



Session 1.5

More IFC for Dummies

Martijn de Riet, Bimforce

Class Description

After two years of IFC related classes in 2013 and 2014, it is again time to take a dive into the world of IFC.

To be able to work with any kind of tool, one must first understand it. Fastest way to do so is reading the manual. Unfortunately, when it comes to IFC there is no manual. Not one written in a language that is understandable for people completely new to the subject.

This class will therefor start with an explanation of IFC in plain English. Where did it come from? Why was it “invented”? By whom, and for what purposes? And, most important of them all: how exactly does it work. How is an IFC model built, defined?

Once we’ve determined the way an IFC is build we can determine why Revit has such difficulties with handling IFC’s. And in what way the translations between Revit and IFC can be improved.

The fact of the matter is that Revit has numerous tools to improve IFC handling. Most of them are well hidden and poorly documented. This class aims to provide these tools. From properly setting up your model, to fully leveraging the Open Source Exporter. This fast paced tutorial will take you along a rollercoaster ride touching all aspects of using IFC in your day-to-day working life:

- How to setup Revit for optimal use of IFC?
- What do all these text files do?
- Can you actually benefit from using IFC?
- What are the most important Do’s and Don’ts when working with IFC?
- Setting up classifications to export to IFC
- Different ways of creating a custom PSet
- Other various enhancements made to the Autodesk Open Source IFC Exporter in the last year.
- What viewers can you use to inspect the exported IFC, and what are their pro’s and cons?

The last subject this class will touch upon is troubleshooting. It’s no secret that Revit and IFC are not a match made in heaven. Even with the handles provided in this class,

things will often go wrong. So what do you do when things go horribly wrong? Are there tools to examine errors, and maybe even to solve them? This class aims to provide a handle on how to systematically examine errors in IFC files and find solutions to the problems at hand. This doesn't mean that after this class you will be able to solve each and every problem. But it will get you through the most often and obvious errors.

In short: this class will give you working knowledge to embrace the full potential of the IFC scheme, improve your exports and create more manageable, data-rich IFC files that can fully leverage the power of IFC: the ability to transfer Information throughout the entire Design, Build and Maintain process.

Martijn de Riet, Bimforce Bimforce

About the Speaker:



Martijn has a degree in building engineering since 2004. His career has been centered on Information Management. Martijn started his career as self-employed BIM engineer and became a well-known Revit expert. In 2015 Martijn co-founded Bimforce, a company aiming to improve communication in the BIM process.

Martijn helped over twenty-five companies implementing BIM workflows and worked as an external BIM Manager on large and complex construction projects, including laboratories, schools, government and research facilities.

Martijn has a history of standard development and a longstanding interest in Open BIM and Open Data workflows. He is main author of the Dutch Revit Standards, and seats in the Technical Committee for the Revit Standards Foundation. Martijn has been Chairman for buildingSMART Benelux from 2015 to 2017, wrote several publications on IFC workflows and has spoken at multiple conferences on the practical implementation of IFC workflows.

Introduction

I started working on developing an understanding of IFC back in 2012. Two years later I thought I was rather good at making it work in Revit. And therefore, rather good at understanding IFC. Now, four more years into studying the OpenBIM language I know what I don't know. And I also know that, objectively speaking, I suck at the whole IFC thing. Maybe somewhat less than the average guy, but still...

But that's ok because, despite some obvious exceptions, we all do. Our entire industry is rooting for a language that only a handful of us are fluent in. Most of us don't understand anything about the grammar or spelling rules of IFC. We don't know (or care) how an IFC is constructed, and what ways you have of properly expressing what's on your mind. We just want it to work.

Even the biggest and most vocal advocates of IFC generally know about the main object-based structure of IFC (you know, a Revit Wall is an IfcWall), how to handle PropertySets (PSetCommon, or custom ones), and perhaps how to use IFC. Although the latter often is mistaken for the ability to import an IFC into another piece of software (like Solibri) and use that software to handle the IFC format.

I do believe that that should be enough. Frankly, the level of insight knowledge required today to effectively leverage the potential added value of IFC in your work process is way too high for IFC to be practical. Think about how it would be if you had to go through the Revit API Documentation¹ for every Family you wanted to build in Revit. That would be ludicrous. Still, that's what we expect people to do when working with IFC. In every IFC Class I teach, I spend at least one hour of making people be somewhat familiar with the IFC Technical Documentation.^{2 3}

This class in no way pretends to be a comprehensive guide (or even dictionary) to IFC. What I do hope to convey is an overview of the things I do know, and a glimpse of the stuff right around the corner. Because I do think IFC is standing on the threshold of a major breakthrough. Which, I know, has been stated for years now. However, things are currently different than they were 5 years ago. On several accounts:

¹ Shoutout to this awesome website: <http://www.revitapidocs.com/2018/>

² Oh dreaded IFC2x3 documentation: <http://www.buildingsmart-tech.org/ifc/IFC2x3/TC1/html/>

³ And slightly less dreaded IFC4 documentation: <http://www.buildingsmart-tech.org/ifc/IFC4/Add2/html/>

1. In the past few years, the amount of people I know using IFC in their day-to-day routine has grown exponentially. In fact, in the Netherlands we have arrived at a place where IFC deliverables for BIM projects are becoming the rule instead of the exception. For a lot of projects, the whole OpenBIM process and results aren't exactly a pretty sight to see, but at least there's more than just a bunch of evangelical loudmouths preaching to the choir.
2. Actual real-world applications start popping up. More and more software is catered towards working with IFC. And actually be pretty darn good along the way too.
3. IFC starts to deliver on some promises. It's not like we can all start a bonfire, ask Kelly Cone to get his ukulele and start singing Kumbaya together, but still. These are pretty exciting times with loads of developments that actually makes IFC work for you. I hope to provide you with some examples to that extent in this class.

Previously in...

As I mentioned earlier, I've been doing research on IFC for a long time now. And to that effect, I've collected and written down a lot of information. I've also been asked repeatedly "When is the IFC for Dummies" book coming out. As always with Open Source stuff, it's the funding that is the problem. Documenting stuff takes a lot of time. And even though I believe some people might be tricked into buying a book, I also think I'm not nearly as convincing as required to pay for the 100's of hours that it would take to index, combine and redact all bits and pieces I have into one single story.

So I've decided to just provide you with an overview of all the stuff I've been doing earlier through our website. If you go to <https://bimforce.com/bedrijf/publicaties/> you'll be able to download all of this content. Unfortunately this part of our website hasn't been translated to English yet, but let's look at it from the bright side:

At least you'll feel familiar looking at some text that you can barely understand... But the downloads are staring you right in the face, so that should be working out just fine. And off course, the information itself is available in English:

- **IFC Mythbuster Part I-IV.** A series of articles I wrote about IFC in general and IFC and Revit in particular.
- **IFC with Revit for Dummies.** A presentation from Autodesk University, which was also recorded. You can watch the recording through the Autodesk website and download all content from our website.
- **IFC for Interoperability.** A presentation given both at RTC Europe and RTC North America about using IFC with Revit. You can download all content, including the presentation, handout and sample files.
- **IFC for Interoperability, pt2.** Follow-up on the first part that provided a more in-depth exploration about IFC with Revit. Again: presentation, handout and sample files have been made available.

I've tried to limit the overlap between this class and previous classes. For that reason, I will not be covering a bunch of things, or refer to this additional documentation to get it over with...

Other resources

Throughout this handout I will be referencing a bunch of online resources. At the end of the handout, I have provided an overview of all different online sources used.

Some of them I've added to the handout too, but I will always reference the original website. Both to give proper recognition and to allow you to keep track of updates and changes to the documents.

I hope you find these helpful. Any feedback is greatly appreciated!

What's IFC exactly?

Well, this has been asked and answered in previous publications. Please refer to those for a detailed explanation of what IFC is, how it's built and how that all works.

In short, the following paragraphs provides my current take on what IFC is and does. Keep in mind though, my understanding of things will shift throughout time. I'm not one of those people that have made their mind up and won't ever question that ever again. So yeah, this definition is roughly the same as provided in earlier publications. But still varies on key aspects. And the same as my ideas have evolved from 2012 to now, they will evolve from here on further.

Which is a fancy way of saying: I don't know either. But I do have some ideas on the subject.

How it came into being⁴

IFC was "invented" as a cooperation between 12 companies:

1. Autodesk
2. Archibus
3. AT&T
4. Carrier Corporation
5. HOK Architects
6. Honeywell
7. Jaros Baum & Bolles
8. Lawrence Berkeley Laboratory
9. Primavera Software

⁴ Read more here: <https://www.buildingsmart.org/about/about-buildingsmart/history/> and watch these on youtube: https://www.youtube.com/watch?v=g_jmGQvr6dQ and <https://www.youtube.com/watch?v=vTyB96O7Xeg>

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

- 10. Softdesk Software
- 11. Timberline Software
- 12. Tishman Construction

This cooperation was triggered because HOK (a user of Autodesk Architectural Desktop) wanted to provide design information linked to a 3D model as a single deliverable for others in the building process to reuse.

For this, they needed an object-oriented and software-independent language that could be easily “harvested” for relevant information for a specific object.

How it works

In numerous of the previous publications and classes I've expanded on the inner workings of IFC⁵. So I won't go there right now, but will instead focus on one important pitfall for using IFC:

It is freaking complicated...

A great visualisation of the complexity of the IFC data schema can be found in a document called “Coordination View v2.0 Wallpaper”.⁶

When you look at the Wallpaper, it basically doesn't make any sense. In one of our research projects from the past few years, we did a complete breakdown on how a single Air Terminal would be defined. As you can see, this makes Revit look like child's play...

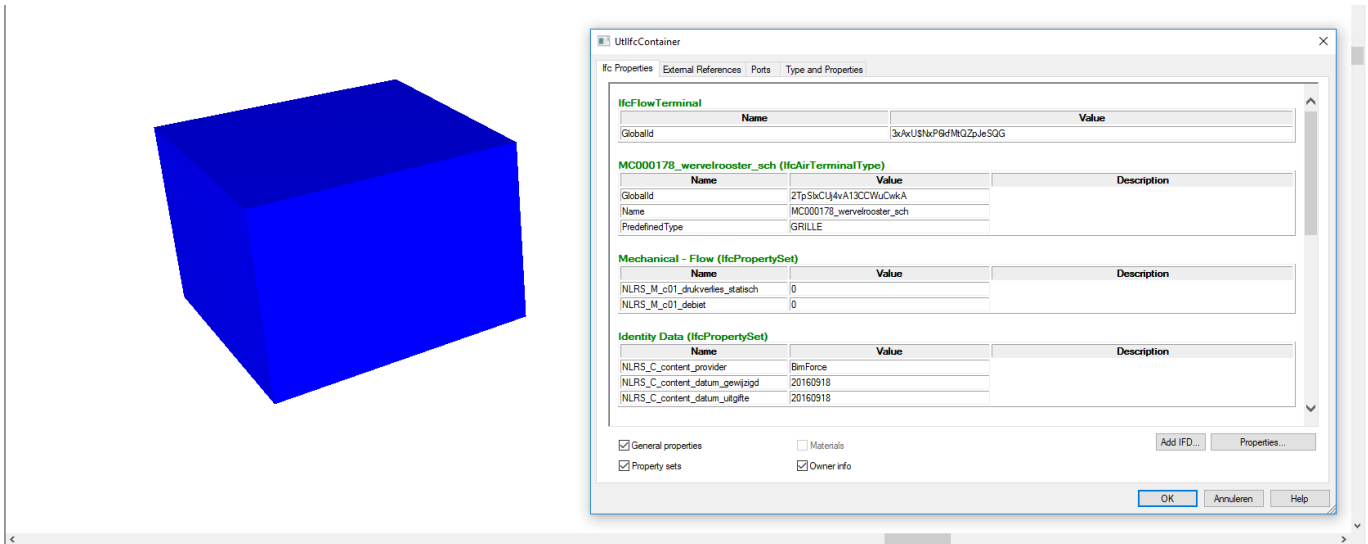
⁵ Besides the resources from our website, you might want to check this documentation out:

<http://www.buildingsmart-tech.org/ifc/>

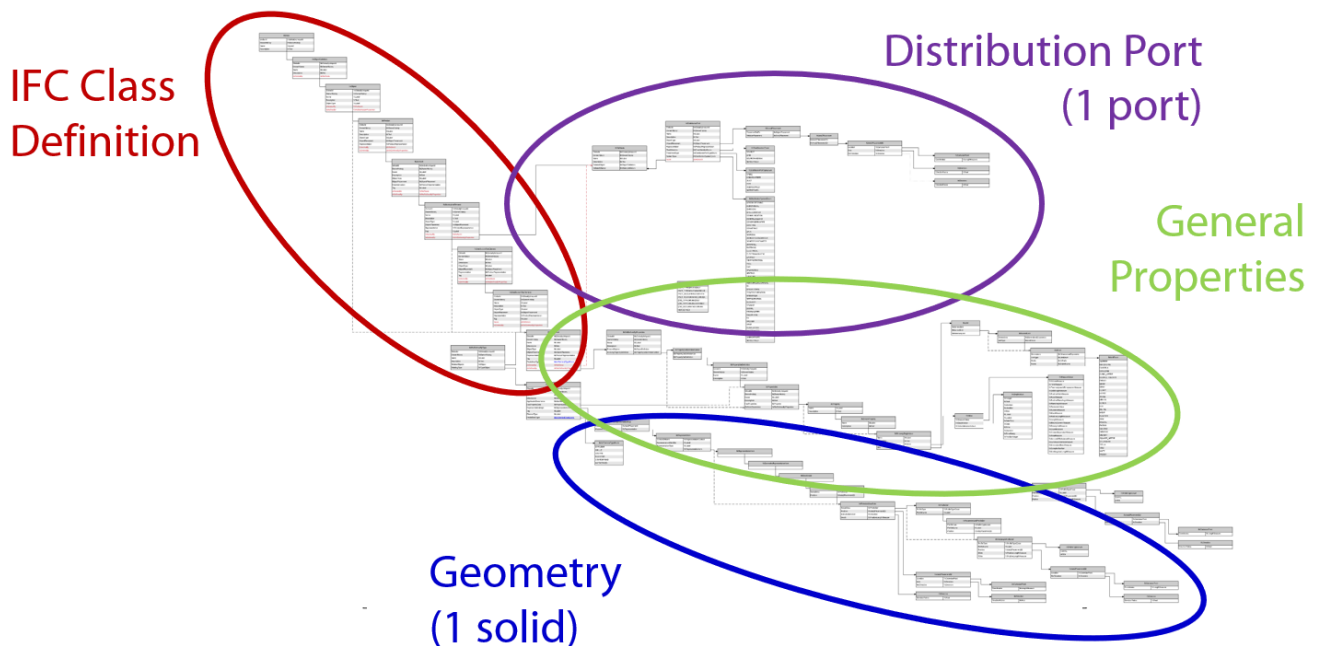
⁶ Find the wallpaper through the following link: http://www.buildingsmart-tech.org/downloads/view-definitions/coordination-view/sub-schema/CoordinationView_V2-0_WallPaper_IFC2X3_Version-1.pdf/view

Martijn de Riet, Bimforce Bimforce

This is the object "in real life":



And this is the corresponding IFC schema to define that object:



IFC Class Definition → This puts the described object somewhere in the IFC hierarchy. In this case, the Class Definition entails everything from IfcRoot to IfcAirTerminal.

Geometry → Any digital 3D object can be broken down into separate parts that describe the 3D object. Think about when you're building a Revit Family. You would

need to define an Extrusion with a sketch, and a height. IFC works / can work in the same way.

General Properties → Everybody (hopefully) knows about the Common PSets. These are basically the same as the Revit Built-In parameters (Description, Type Mark, and so on). Only more. Way more... Besides the PSet_Common there are roughly 15 standard PSets available (depending on which Class you use). When considering the IfcAirTerminal, the following standard PSets can be used:

1. Pset_AirTerminalOccurrence
2. Pset_AirTerminalTypeCommon
3. Pset_SoundGeneration
4. Pset_ElectricalDeviceCommon
5. Pset_EnvironmentalImpactIndicators
6. Pset_EnvironmentalImpactValues
7. Pset_Condition
8. Pset_ManufacturerOccurrence
9. Pset_ManufacturerTypeInfoInformation
10. Pset_ServiceLife
11. Pset_Warranty

Distribution Ports → Distribution Ports can be considered to be Revit Connectors on some serious steroids. While a Revit Connector has some basic functionality in terms of passing through information from one Revit Element to another (e.g. Flow, direction, and so on), IFC Distribution Ports are:

- a. available in way more types then “just” Electrical, Mechanical and Plumbing
- b. capable of passing through way more information through their (standard) properties.

What's not here? → Well, to start with the above sample does NOT contain any material definition. But don't worry, that's only the most complicated part... Imagine a fusion from hell between Distribution Ports and Geometry Definitions and you have a pretty good idea of what's still missing.

Anything else? → Besides missing the material definition, this is vanilla IFC, pure and simple (in the most metaphorical way possible...). When you really want to re-use an IFC component, the IFC definition should, and can, also contain “instructions” on how to properly translate the IFC definition. So the Properties section can contain something called “External References” to define how an IFC Property should translate into a Revit Parameter (including GUID, datatype, unittype, parameter group, and so on).

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

Conclusion

So it's complex. Really, really complex.

The big caveat now is: how do we make it usable for mere mortals? Let alone make it usable for Dummies?

To start with: let's look at how we can set up Revit properly in the next chapter. Then let's look at some alternatives for IFC viewers that you can use.

And to top it all off: let's look at how things would work out when you don't even know you're using IFC... Which, in all fairness, is really the way things should be heading. The only way a Dummie would be able to work with IFC properly is if he/she doesn't even know it.

How to setup Revit for optimal use of IFC?

Introduction

When working on IFC with Revit, get ready to go seriously old-school. Unfortunately, most of the configuration for IFC with Revit is done in simple (technologically speaking that is) text files.

Which is an absolute disgrace off course, but something we have to live with. This chapter will therefore be largely about how to work with those text files.

Besides that, a large chunk will be aimed at providing you with simple (modelling) guidelines on working with IFC.

What do all these text files do?

When working on IFC with Revit, get ready to go seriously old-school. Unfortunately, most of the configuration for IFC with Revit is done in simple (technologically speaking that is) text files. How to work with these textfiles is well document in the Revit IFC Handbook by Autodesk (see the resources for a downloadlink, or find it added to this handout.

This paragraph will share the most important summaries on the purpose and how-to's on using these files.

Import Mapping File

When you import an IFC file, you need to use an Import Mapping Table to properly translate IFC Classes to Revit Categories. The import mapping table does just that. There are a few things to keep in mind though:

1. The textfile is spelling and case sensitive. Tedious vetting is required to make sure you don't make any mistakes (see the following paragraph for an example of stuff going wrong when you make an error here.
2. Make sure you use a specific IFC template when importing (or linking) IFC's. This template should hold standard IFC Shared Parameters and mapping table.
3. Even though they're present in the Import Mapping Table, most System Families are hardcoded into the importer. Which means that an IfcWall will be a Revit wall, no matter what you put in the file.

An example of this table, created for the Dutch Revit Standards (NLRs) is added to this handout.

Export Mapping File

When you export an IFC file, you need to use an Export Mapping Table to properly translate IFC Classes to Revit Categories. The import mapping table does just that. There are a few things to keep in mind though:

1. The textfile is spelling and case sensitive. Tedious vetting is required to make sure you don't make any mistakes (see the following paragraph for an example of stuff going wrong when you make an error here).
2. Make sure you use a specific IFC template when importing (or linking) IFC's. This template should hold standard IFC Shared Parameters and mapping table.
3. Even though they're present in the Export Mapping Table, most System Families are hardcoded into the exporter. Which means that a Revit Wall will be an IfcWall, no matter what you put in the file. Ok, not completely true, you can also tell a Revit Wall to be an IfcFooting. But that's about it. For an overview of the possible mappings for System Families, see the screenshot below taken from the Autodesk Revit IFC Handbook

Revit K Category	Default IFC class	Predefined Type	Alternative classes (IfcExportAs)	Alternative types (IfcExportType)
Wall	<i>IfcWallStandardCase</i>	UNDEFINED	<i>IfcFooting</i>	PAD_FOOTING PILE_CAP STRIP_FOOTING FOOTING_BEAM
	<i>IfcWall</i>	STANDARD	<i>IfcFooting</i>	
Slab	<i>IfcSlab</i>	FLOOR		FLOOR ROOF LANDING BASESLAB
			<i>IfcFooting</i>	PAD_FOOTING PILE_CAP STRIP_FOOTING FOOTING_BEAM
			<i>IfcCovering</i>	CEILING FLOORING CLADDING ROOFING
			<i>IfcRamp</i>	
Ceiling	<i>IfcCovering</i>	-	-	CEILING FLOORING CLADDING ROOFING
Ramp	<i>IfcRamp</i>	-	-	-
Stairs	<i>IfcStair</i>	-	-	-

Additionally, see the below screenshot for an example of how easy it is to screw up the mapping. This image is from an officially published Export Mapping Table (part of the NLRsv2.5.2). Needless to say most floors won't get exported...

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

Flex Ducts	Clearance Zones	IfcBuildingElementProxyType	USERDEFINED
Flex Ducts	Pattern	IfcAnnotation	
Flex Pipes		IfcPipeSegmentType	FLEXIBLESEGMENT
Flex Pipes	Center Line	IfcAnnotation	
Flex Pipes	Clearance Zones	IfcBuildingElementProxyType	USERDEFINED
Flex Pipes	Pattern	IfcAnnotation	
Floors	IfsSlab		
Floors	Bounding Boxes	IfsSlab	
Floors	Clearance Zones	IfcBuildingElementProxyType	USERDEFINED
Floors	Common Edges	Not Exported	
Floors	Connection Zones	IfcBuildingElementProxyType	USERDEFINED
Floors	Cut Pattern	IfcAnnotation	
Floors	Finish 1 [4]	IfcSlab	
Floors	Finish 2 [5]	IfcCoveringType	FLOORING
Floors	Hidden Lines	Not Exported	
Floors	Interior Edges	Not Exported	
Floors	Maintenance Zones	IfcBuildingElementProxyType	USERDEFINED
Floors	Membrane Layer	IfsSlab	
Floors	Placement Zones	IfcBuildingElementProxyType	USERDEFINED
Floors	Slab Edges	IfcMemberType	USERDEFINED
Floors	Structure [1]	IfsSlab	
Floors	Substrate [2]	IfsSlab	
Floors	Surface Pattern	IfcAnnotation	
Floors	Thermal/Air Layer [3]	IfsSlab	
Furniture		IfcFurniture	

An (correct) example of this table, created for the Dutch Revit Standards (NLRs) is added to this handout.

Autodesk IFC Shared Parameters

Autodesk uses a specific set of IFC Shared Parameters for working with IFC. Partly these parameters are used to improve the Export Mapping (IfcExportAs, IfcExportType). Partly they can be used to improve IFC datastructure (IfcSystem, IfcZone, IfcName, IfcLongname, IfcDescription, IfcTag) and partly they provide a central parameter definition for IfcCommon properties. Using these Shared Parameters in your Revit model negates the need for a Parameter Mapping File (that is, unless you need an IFC parameter that's not in this list, and if you're willing/able to work with English parameter names).

I've added the Shared Parameter File to the handout. Which immediately poses an issue:

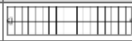
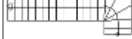
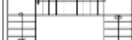
Recently Autodesk published an IFC Manual (referenced elsewhere in this document and added to the Dataset). With this manual, an IFC Shared Parameter file has been published (*IFC Shared Parameters.txt*, also in the handout Dataset).

However, on the Sourceforge website, two different files have been published (*IFC Shared Parameters File 3-10-15.txt* and *IFC Shared ParametersTypes 3-10-15.txt*. Both also added to the Dataset).

As you might have anticipated: those sets of Shared Parameters do not match. It seems like the Autodesk guys that made the Handbook were missing the Types file published on the Sourceforge website. I checked, but the missing parameters still seem to work. I would advise you to use the Sourceforge version...

So what do these standard parameters do? It will be too much to give you an overview of all these parameters, so I'll stick to the most important ones:

1. **IfcExportAs.** The *IfcExportAs* parameter allows you to manually override the *IfcClass* for any loadable family. It also allows you to apply certain overrides to System Families (see the Export Mapping Table paragraph for specifics).
2. **IfcExportType.** Like Revit, IFC knows "types". However, they are slightly different. *IfcTypeEnumerations* (as they call them) are basically specific subsets of an *IfcClass*. It has nothing to do with parameters, or parameter values. It has everything to do with real-life use cases. For instance, the *IfcStair* has several Type Enumerations:

Enumerator	Description	Figure
StraightRunStair	A stair extending from one level to another without turns or winders. The stair consists of one straight flight.	
TwoStraightRunStair	A straight stair consisting of two straight flights without turns but with one landing.	
QuarterWindingStair	A stair consisting of one flight with a quarter winder, which is making a 90° turn. The direction of the turn is determined by the walking line.	
QuarterTurnStair	A stair making a 90° turn, consisting of two straight flights connected by a quarter space landing. The direction of the turn is determined by the walking line.	
HalfWindingStair	A stair consisting of one flight with one half winder, which makes a 180° turn. The orientation of the turn is determined by the walking line.	
HalfTurnStair	A stair making a 180° turn, consisting of two straight flights connected by a half space landing. The orientation of the turn is determined by the walking line.	
TwoQuarterWindingStair	A stair consisting of one flight with two quarter winders, which make a 90° turn. The stair makes a 180° turn. The direction of the turn is determined by the walking line.	
TwoQuarterTurnStair	A stair making a 180° turn, consisting of three straight flights connected by two quarter space landings. The direction of the turn is determined by the walking line.	
ThreeQuarterWindingStair	A stair consisting of one flight with three quarter winders, which make a 90° turn. The stair makes a 270° turn. The direction of the turn is determined by the walking line.	
ThreeQuarterTurnStair	A stair making a 270° turn, consisting of four straight flights connected by three quarter space landings. The direction of the turn is determined by the walking line.	
SpiralStair	A stair constructed with winders around a circular riser often without landings. Depending on outer boundary it can be either a circular, elliptical or rectangular spiral stair. The orientation of the winding stairs is determined by the walking line.	
DoubleRunStair	A stair having one straight flight to a wide quarter space landing, and two side flights from that landing into opposite directions. The stair is making a 90° turn. The direction of traffic is determined by the walking line.	
CurvedRunStair	A stair extending from one level to another without turns or winders. The stair is consisting of one curved flight.	
TwoCurvedRunStair	A curved stair consisting of two curved flights without turns but with one landing.	
UserDefined	Free form stair (user defined operation type)	
NotDefined		

You can add a Type Enumeration to the IfcExportAs parameter in this syntax: IfcStair.SPIRALSTAIR, but I've always preferred keeping the two separated.

- IfcName.** Allows you to define/override Name attribute for object instances and type objects. More on that later.
- IfcLongName.** Allows you to define/override LongName attribute for spatial object instances. This is helpful because IFC does not support the concept of Numbering elements (Room Number, Project Number, etc). If you use IfcName as a Number, you can use IfcLongName as the actual name.
- IfcDescription.** Allows you to add a description to the ObjectInstance. Much like the Description parameter in Revit.
- IfcSystem.** Allows you to override the System Name value that the IFC Exporter will assign to MEP systems upon export.
- ZoneName / ZoneDescription.** Allows you to create Spatial Zones (e.g. fire compartments, space usage, and so on). If you need more than one Zone type, just create more parameters with an incremental number added (ZoneName1, ZoneName2, and so on)

- 8. IfcObjectType.** Allows you to define/override ObjectType attribute for object instances. Not quite sure when you would need this since the IfcObjectType is a quite high-level attribute.
- 9. IfcElementType.** Allows you to define ElementType for type objects. Again, not seeing a direct usecase for this one.
- 10. IfcTag.** Allows you to define Tag attribute for object instances and type objects
- 11. IfcApplicableOccurrence.** Allows you to to define ApplicableOccurrence attribute for type objects.
- 12. IFC CAD Layer.** Allows you to set an override to the CAD Layer the exporter would use. For more information about setting up, and assigning proper CAD standards upon export to IFC, check the Autodesk IFC Manual.

There are loads more IFC Shared Parameters, but they tend to be part of the IfcCommon properties so I'll leave them be for now.

If you're not willing to use the standardised Autodesk IFC Shared Parameters, or require more information to into the IFC file, the following subjects might interest you:

Parameter Mapping File

Let's say you have your own (Shared) Parameters to use and want to use these in IFC. The first thing you need to do is check whether your Shared Parameters match so called IfcCommon parameters. IfcCommon parameters can best be described as the Out of the Box parameters for IFC. Just like Revit has standard parameters for IFC (Description, Function, Is Structural, Phase, basically all Properties that you see when starting a blank Family from template), IFC has them too. They are called the Common Properties, and they're different for each IFC Class.

For instance, IfcWall has the following PSet_WallCommon defined:

Property	IFC2x3	IFC4	Revit ootb	ADKS IFC SP
Reference	yes	yes	Type Name	yes
Status		yes		
AcousticRating	yes	yes		yes
FireRating	yes	yes	Fire Rating	Fire Rating
Combustible	yes	yes		yes
SurfaceSpreadOfFlame	yes	yes		yes
ThermalTransmittance	yes	yes	U-Value	yes
IsExternal	yes	yes	Function	yes

LoadBearing	yes	yes	Structural	yes
ExtendToStructure	yes	yes	Top Is Attached	yes
Compartmentation	yes	yes		yes

In the ParameterMappingTable file, which I added to this handout, the mapping is made as follows:

<i>Pset_WallCommon</i>	<i>FireRating</i>	<i>NLRS_C_brandwerendheid</i>
<i>Pset_WallCommon</i>	<i>IsExternal</i>	<i>Function</i>
<i>Pset_WallCommon</i>	<i>LoadBearing</i>	<i>Structural</i>
<i>Pset_WallCommon</i>	<i>Reference</i>	<i>Description</i>
<i>Pset_WallCommon</i>	<i>ExtendToStructure</i>	<i>Top is Attached</i>
<i>Pset_WallCommon</i>	<i>AcousticRating</i>	<i>NLRS_C_geluidwering_Rw</i>
<i>Pset_WallCommon</i>	<i>Combustible</i>	<i>NLRS_C_brandbaar</i>
<i>Pset_WallCommon</i>	<i>SurfaceSpreadOfFlame</i>	<i>NLRS_C_code_brandklasse</i>
<i>Pset_WallCommon</i>	<i>Compartmentation</i>	<i>NLRS_C_compartimentering</i>

Some notes:

1. The ThermalTransmittance parameter isn't mapped. This is because this mapping process has regular unit translation issues, while ootb export functionality seems to function properly.
2. The Status parameter isn't mapped. Mapping IFC2x3 and IFC4 in one file can cause a crash.
3. The file attached to the handout is not complete, and based on the NLRS. You might need to remap some parameters to your own standard of choice.

Custom PropertySet

So you've added the Labels. You've used the IFC Parameters (or mapped your own to the PSet_Common propertysets) but still feel the need to export additional information to IFC.

In this case, you can use Custom Propertysets. These are user-defined datasets, much like you would create your own parameters in Revit.

More information on how they work later on in this handout, where we will setup a simple Custom Propertyset.

IFC Tips, Tricks and NoNo's

Edit Profile & Attach to...

I know..., me neither.

I'm one of those guys that hates the use of "modelling shortcuts". Or hacks, as I like to call them. Because that's what Autodesk devs allow you to do: hack the system. And nothing say "I kinda cheated" more then having your IFC Export go to hell because you took some shortcuts.

One way to avoid an embarrassing model coordination session where random stuff just isn't showing up or grossly disfigured, is to simply not use things like Edit Profile. That would be best for everyone in my humble opinion. But if you do chose to do so, there's some rules to take into account:

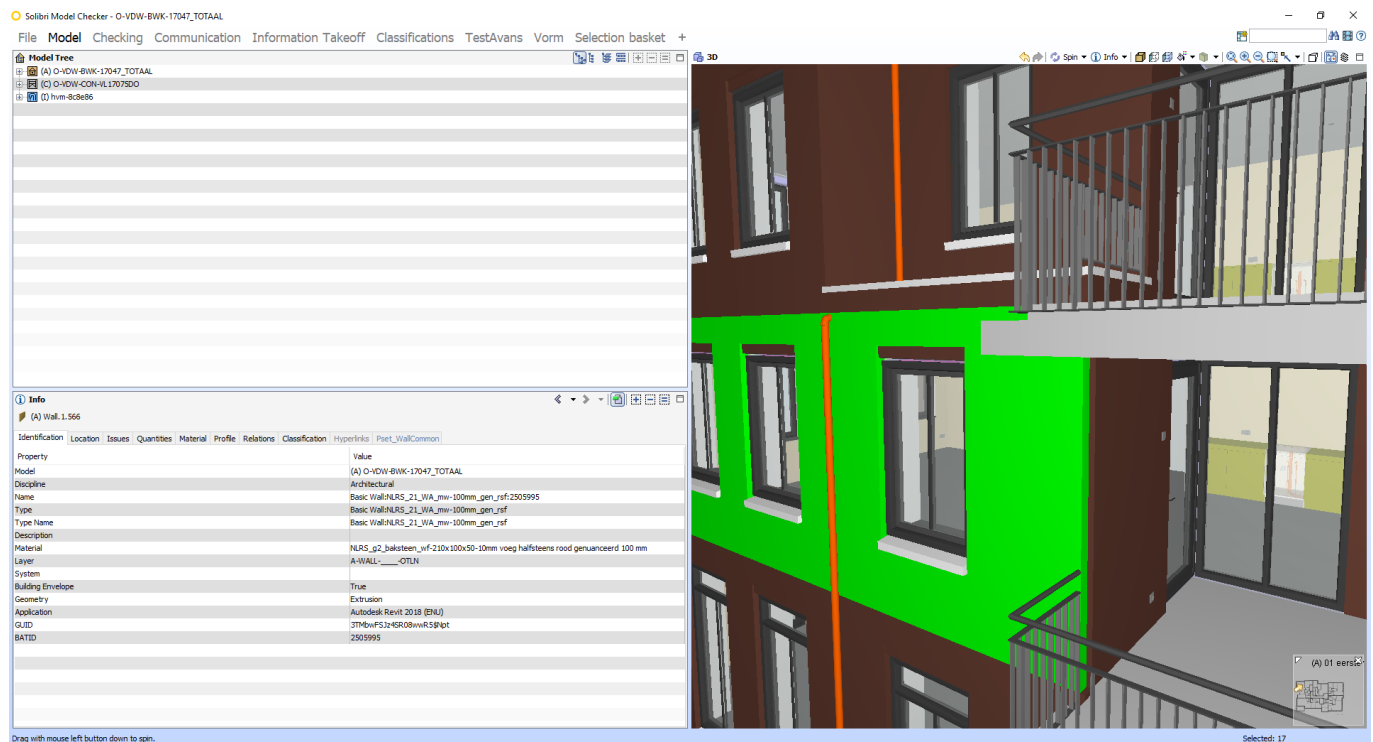
1. Cut, don't paste. By far the most issues with non-complying geometry is based on the fact that people will use Edit Profile and Attach To functionality in a way that adds geometry. This is not good. If you feel the need to have Revit deal with keeping your Walls modelled correctly when you start goofing around with the Roof: make sure that the detached wall is higher than the attached wall. The IFC Exporter has more difficulties with adding geometry that isn't really there then carving it off.

2. Be gentle with Wall Joins. Same goes with joined walls. If you have a Wall in Revit which has an Edited Profile, chances are the Wall joins will not work out. In those cases you should use Disallow Wall Join to manually model the correct joint situation yourself.

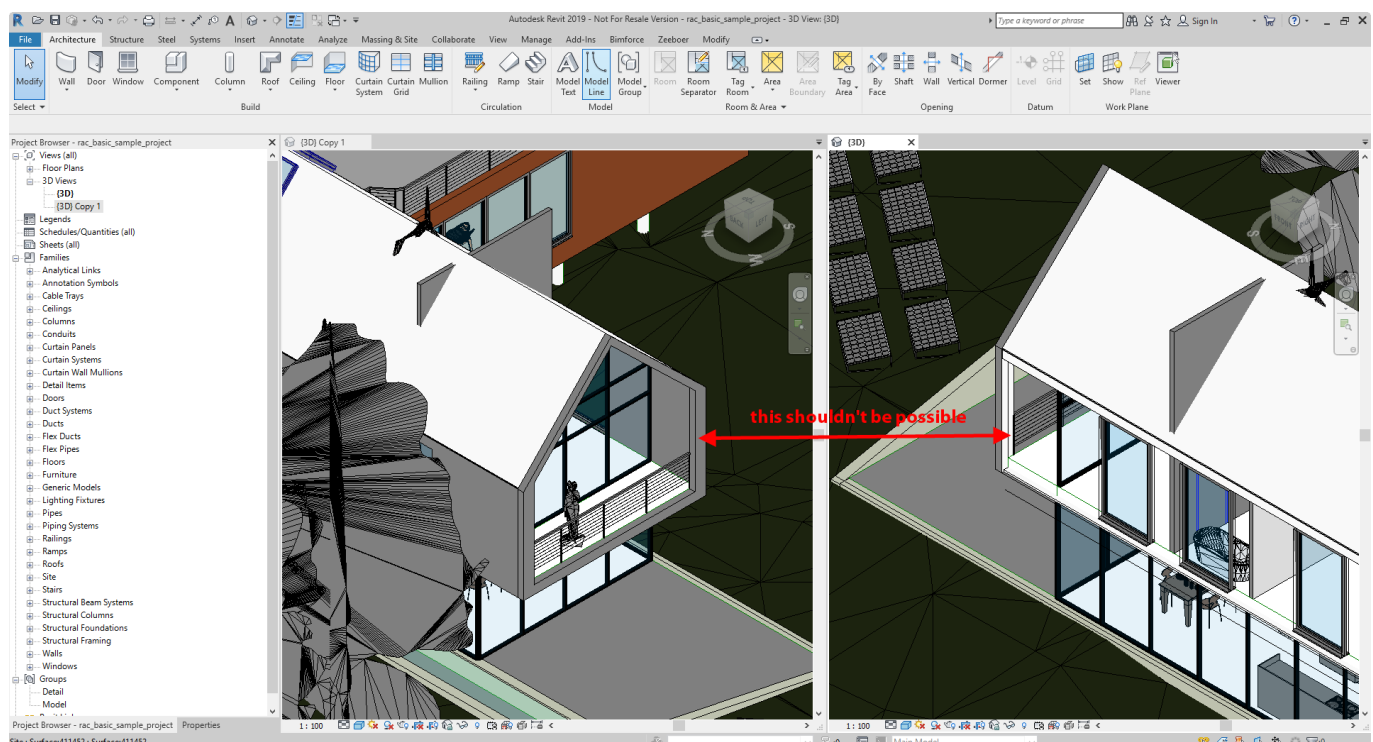
The following image shows a Wall with an edited profile. The join with the other brick wall isn't reproduced in IFC.

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce



3. But seriously: don't... One of the most striking examples of Revit going haywire when importing an IFC with goofy geometry is the Basic Sample File. Didn't see that one coming now did you?



It's an interesting showcase for multiple reasons. For one it's the official Autodesk Sample Model. But besides that, the bug is not even possible to achieve in Revit. Having any kind of object that is transparent from one side, but solid from the other? That would be need.

Unfortunately, there is no documentation on how this visualisation feature can be used at will.

Cutting the corner

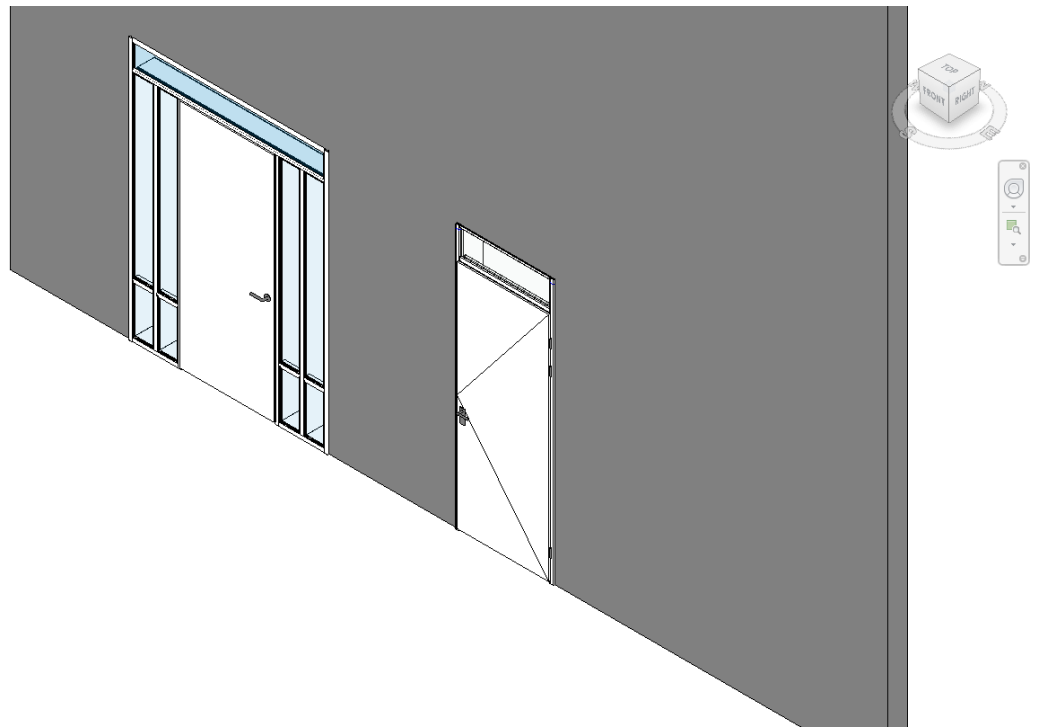
One of the major issues of working with Revit and IFC is that sometimes your IFC file just explodes in size. Now, we will be talking about how to figure out which part of your Revit model is responsible for this.

For now, the important thing is to remember:

1. It's one thing to use complex, rounded geometry profiles in Revit. But for the love of god, don't make them go around the corner. As a rule of thumb: the Revit IFC Exporter doesn't mind detailed geometry, as long as it is just an Extrusion. If you're doing these type of profiles and make them go around the corner, things go wrong.

As an example, I would like to show you a file with two door families in them, from two different dutch Door Manufacturers, Svedex and Berkvens. I've added this file to the handout (Test_cutting_corners.rvt).

As you can see in the following image, they're similar in terms of level of geometric detail. The only difference is: the Svedex doors have been modelled "rounding corners", which results in an IFC file of a staggering 41mb! The Berkvens Doors, which are modelled with just straight extrusions work properly and only result in a 117kb IFC file.



Now, it needs to be mentioned that Svedex asked us to change their library to provide with better IFC support. Currently, their Doors will export in a similar matter as the Berkvens Doors.

Shared Nested Families, Groups and Assemblies

This one is thoroughly covered in the Autodesk Revit IFC Manual. I've added the manual to the handout.⁷ Please read up on the matter there. In short summary, these are the rules you need to take into account:

1. Shared Nested Components will be exported to IFC as separate elements. The sub-elements have no connection to the original component. If you still want to recognise them in the IFC file, you would need to create a identifier parameter in all subparts that are linked to the main component.

⁷ The Autodesk Revit IFC Manual can also be found here:

http://abcbdblog.typepad.com/abcd/2018/Success_Stories/IFC-Manual-2018-ENU.PDF

2. Groups are recognized in IFC. The items in the group will still be available separately, but will also be available as an IfcGroup. So you can also view all those elements together. Only downside is that an IfcGroup is shown completely at the top of the Ifc Hierarchy Tree, and not at the level that the elements are on.

3. Assemblies are also recognized in IFC. The items in the group will still be available separately, but will also be available as an IfcAssembly. So you can also view all those elements together. The difference with Groups is, besides the Revit modelling part, that Assemblies are part of the IFC Hierarchy tree (they will show up on the correct Level in a tree view).

Overriding the Project, Site and Building IfcGuid

If you're working on a project that exists of multiple Revit models (for instance Architectural, Structural and MEP), it is preferable to have the same GUIDs for all models. This way any software importing the IFC files "knows" they belong together.

There's basically two ways of doing this:

1. Use the same project file to start with.

Open a project from your template and provide this to all parties. The Project-, Site- and Building Guides are created when you start a new project. So this way all parties use the same Guides. This is a fairly simple and straightforward solution, which has one pitfall: For this to work, the template file would have to be a combined template that all parties can use. That's often not the case, most companies have their own Revit templates to work with.

2. Override the Guides using Autodesk IFC Parameters.

This workflow was discovered by Dirk van Rillaer, Rijksvastgoedbedrijf.⁸

Add three parameters to your Project Information, as shown below. Fill these out with properly formatted, but bogus GUIDs, and export to IFC. For the first export, uncheck the tickmark in the exporter that says "Store the IFC Guid in an element parameter after export". After the first export, you can turn it back on. At that point the GUIDs stored in the file will no longer be overwritten.

⁸ Dirk posted the workflow here:

<https://sourceforge.net/p/ifcexporter/discussion/general/thread/634a2a16/>

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

Project Information

Family: System Family: Project Information Load...

Type: Edit Type...

Instance Parameters - Control selected or to-be-created instance

Parameter	Value
Identity Data	
Organization Name	Organization Name
Organization Description	Organization Description
Building Name	Building Name
Author	Author
Energy Analysis	
Energy Settings	Edit...
IFC Parameters	
IfcApplicableOccurrence	IfcApplicableOccurrence
IfcName	IfcName
IfcTag	IfcTag
IfcBuilding GUID	aAbBcCdDeEfGhHijJ\$B
IfcSite GUID	aAbBcCdDeEfGhHijJ\$\$
IfcProject GUID	aAbBcCdDeEfGhHijJ\$P
Other	
Project Issue Date	Issue Date
Project Status	Project Status
Client Name	Owner
Project Address	Enter address here
Project Name	Project Name
Project Number	0001

OK Cancel

Modify Setup

<In-Session Setup>

<IFC2x3 Coordination View 2.0 Setup>

<IFC2x3 Coordination View Setup>

<IFC2x3 GSA Concept Design BIM 2010 Setup>

<IFC2x3 Basic FM Handover View Setup>

<IFC2x2 Coordination View Setup>

<IFC2x2 Singapore BCA e-Plan Check Setup>

<IFC2x3 COBie 2.4 Design Deliverable Setup>

<IFC4 Reference View Setup>

<IFC4 Design Transfer View Setup>

General Additional Content Property Sets Level of Detail Advanced

☐ Export parts as building elements

☐ Allow use of mixed "Solid Model" representation

☐ Use active view when creating geometry

☐ Use family and type name for reference

☐ Use 2D room boundaries for room volume

☐ Store the IFC GUID in an element parameter after export

☐ Keep Tessellated Geometry as Triangulation

OK Cancel

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

IFC Structure

Active	Type	Name	Description
☒	Project	0001	
☒	Site	Default	
☒	Building	Building Name	
☒	Building Storey	Level 1	

Properties

Location

Classification

Name	Value	Unit
Element Specific		
FileName	Test Ifc Guid override_1.ifc	
Guid	aAbBcCdDeEfFgGhHiIjJkP	
Identity	IfcProject	
LongName	Project Name	
Name	0001	
Phase	Project Status	
File Header		
Description	ViewDefinition [CoordinationView_V2.0]	
Implementation Level	2;1	
Originating System	20180328_1600(x64) - Exporter 19.1.0.0 - Alternate UI 19.1.0.0	
Preprocessor Version	The EXPRESS Data Manager Version 5.02.0100.07 : 28 Aug 2013	
Schema Identifiers	IFC2X3	
Time Stamp	2018-10-11T08:25:15	

Dynamo and IFC workflows

So there's Dynamo. Which allows you to automate certain tasks. Basically, Dynamo works on the Revit API and can do most of the stuff you can use the API for (mind you, not everything).

I've been very ambivalent towards Dynamo. I love the concept of visual coding because it allows non-programmers to still leverage the possibilities of custom built software, without costing a large amount of money.

However, as a software developer I also know that most of the time involved with creating software is **not** the actual functionality. It's all stuff around the functionality: Quality Assurance and testing procedures, User Interface, security, user management,

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

error handling, documentation, the list goes on and on. People don't realise this (as I didn't realise this when I started venturing in software development). And it is exactly this part that is missing completely with Dynamo. You're just looking at, and dabbling with, the naked source code. Literally anyone can change (and screw up) the dynamo scripts. And there's no way you can figure out who's responsible for the giant clusterfuck you get left behind with. And this is why I always advice my clients (and now you) the following:

Use Dynamo. Use it to discover what you can do, to prototype solutions for your problems. But after that's done you do not want Dynamo in your production workflow because of the risks involved. Pick up your Dynamo scripts, take them to a software developer and ask him to "make a button". But then with all the stuff that comes with a serious software development.

Having said all that, there have been some interesting developments on using Dynamo with IFC. In this section I will be discussing two of them. Since I'm not fluent with Dynamo myself, and these aren't my routines, I'll leave it up to you to see if you can benefit from them.

1. Materials to Parts

One of the issues with Parts and IFC is the inability to map Materials to Parts. So with the standard (or Open Source) Exporter, the Parts will have no recognisable Materials added.

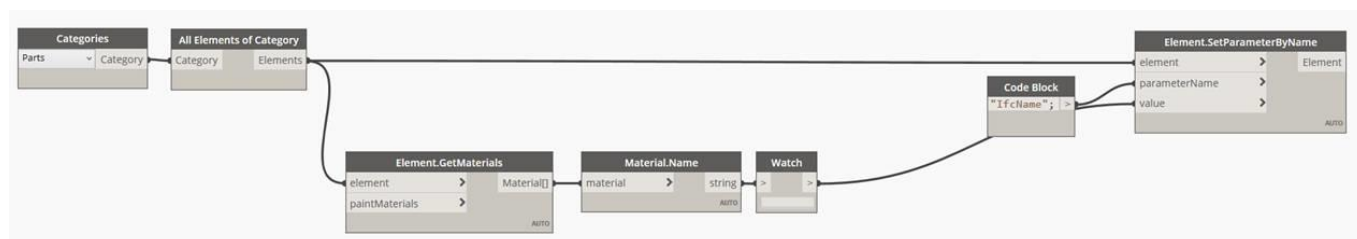
		Material Layer	Gypsum Wall Board
✓	✓	Wall	Basic Wall:Exterior - Render on Brick on Block
		Wall Type	Basic Wall:Exterior - Render on Brick on Block
✓	✓	+ Building Element Part	Part
✓	✓	+ Building Element Part	Part
✓	✓	+ Building Element Part	Part
✓	✓	+ Building Element Part	Part
✓	✓	+ Building Element Part	Part
✓	✓	+ Building Element Part	415
✓	✓	+ Building Element Part	Part
✓	✓	+ Building Element Part	414

To get the Parts exported properly, you would to manually reassign all Materials. A simpler, and faster, way to do this is by using Dynamo for renaming all Materials.

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

The following Dynamo script has been developed by Dirk van Rillaer, Rijksvastgoedbedrijf. And he was kind enough to share it with me for this class. I've added the Dynamo script to the handout. The below image shows how it works.



The outcome is a nicely mapped Material naming convention in the resulting IFC, as shown below.

IFC Structure			
Active	Type	Name	
☒	Project	0001	
☒	Site	Default	
☒	Building		
☒	Building Storey	Level 1	
☒	Walls		
☒	Standard Wall	Basic Wall:Exterior - Render on Brick on Block	
☒	Wall	Basic Wall:Exterior - Render on Brick on Block	
	Wall Type	Basic Wall:Exterior - Render on Brick on Block	
☒	Building Element Part	EIFS, Exterior Insulation	
☒	Building Element Part	Render, Tan, Textured	
☒	Building Element Part	Render, Tan, Textured	
☒	Building Element Part	Cavity Fill	
☒	Building Element Part	Gypsum Wall Board	
☒	Building Element Part	Concrete Masonry Units	
☒	Building Element Part	Air	
☒	Building Element Part	Carpet	
☒	Slabs		
☒	Slab	Floor:Concrete-Domestic 425mm	
☒	Slab	Floor:Concrete-Domestic 425mm	
	Slab Type	Floor:Concrete-Domestic 425mm	
☒	Building Element Part	Concrete, Sand/Cement Screed	
☒	Building Element Part	Concrete, Cast In Situ	
☒	Building Element Part	Rigid insulation	
☒	Building Element Part	Site - Hardcore	

2. Fill out IfcName overrides based on automation rules

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

One of the major irritations with people is the way the IFC Exporter generates a name for the object. The syntax is as follows:

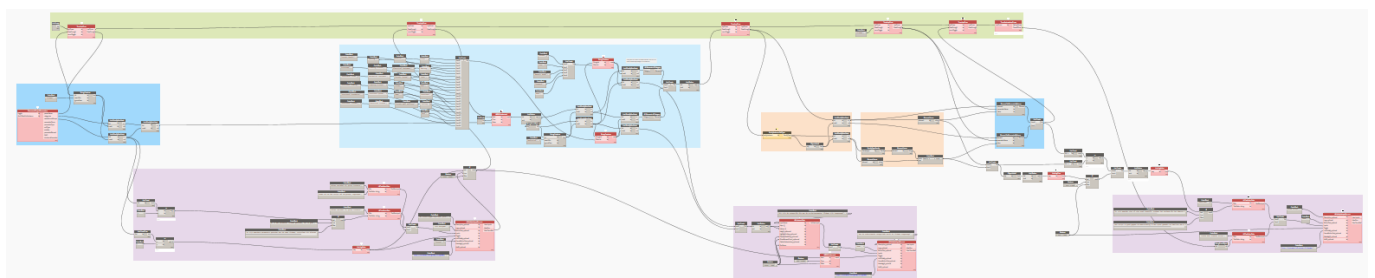
FamilyName:TypeName:RevitID

Name	2E(32.3)_Svedex_Match_VO3LR:2E(32.3)_Svedex_Match_VO3LR_2315_930_2630_2000:300948
ObjectType	2E(32.3)_Svedex_Match_VO3LR:2E(32.3)_Svedex_Match_VO3LR_2315_930_2630_2000

Which means that **every single object** in the resulting IFC file has a unique name. This can be a real pain if you want to find out how many Doors of a certain type you have in your project.

I have been discussing the use of the Autodesk IFC Parameters earlier. In the previous paragraph we used them to unify the IfcGuids for your Project, Building and Site. You can do the same with other parameters, including the IfcName. Now, you can imagine that it's a real pain to manually fill out all IfcName overrides in your entire project.

So some really smart guys at HFB Groep (<https://www.hfb-groep.nl/?lang=en>) built a Dynamo script that automates this. I've added the script to the handout. You can build from here to also support other additional data and overrides such as IfcDescription, IfcTag and others.



Open Source Exporter developments

In this chapter we've covered a lot of stuff that has been developed or researched for the Autodesk Open Source IFC Exporter. However, there are some other developments

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

which have just started. They may not be robust enough to be part of your day-to-day workflow, but they do provide some interesting insights and (limited) application.

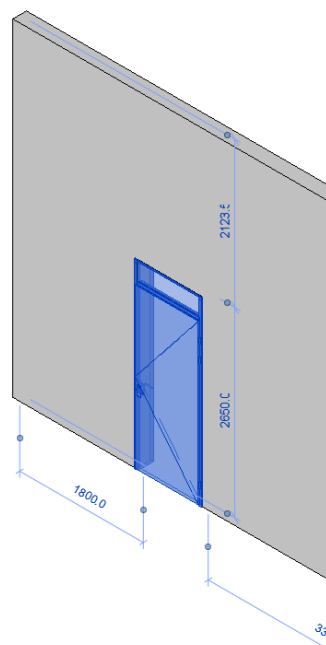
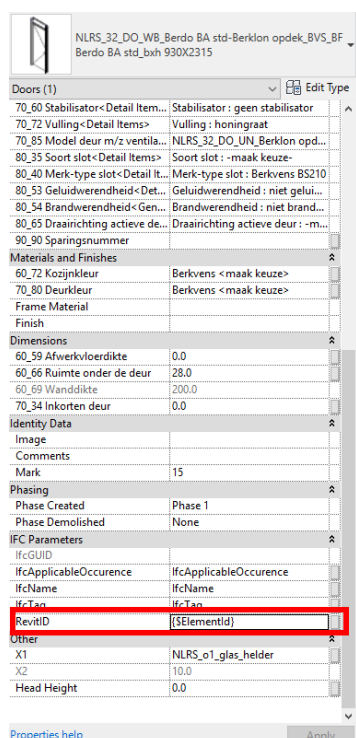
Using Expressions

A fairly new addition to the Ifc Exporter project has been the use of "Expressions". These allow you to add calculations to your property values upon export to IFC.⁹ I haven't found the time yet to discover a lot of usecases (but I'm pretty sure there are). But obvious usecase I can already give you, and that is getting the RevitID into an IFC Property.

Since we learned that we can:

- Override IfcName (which doesn't show the Revit ID anymore) and
- Override IfcTag (which is where Revit automatically places the ID)

We kind of broke the whole RevitID export workflow. This can be fixed by adding a new parameter to the Revit element and filling in an expression as shown below. If you also add this parameter to your Custom PSet, this will export properly.



⁹ The functionality is briefly described here:

<https://sourceforge.net/p/ifcexporter/wiki/Notes%20on%20parameter%20expression/>

1.5 - More IFC for Dummies

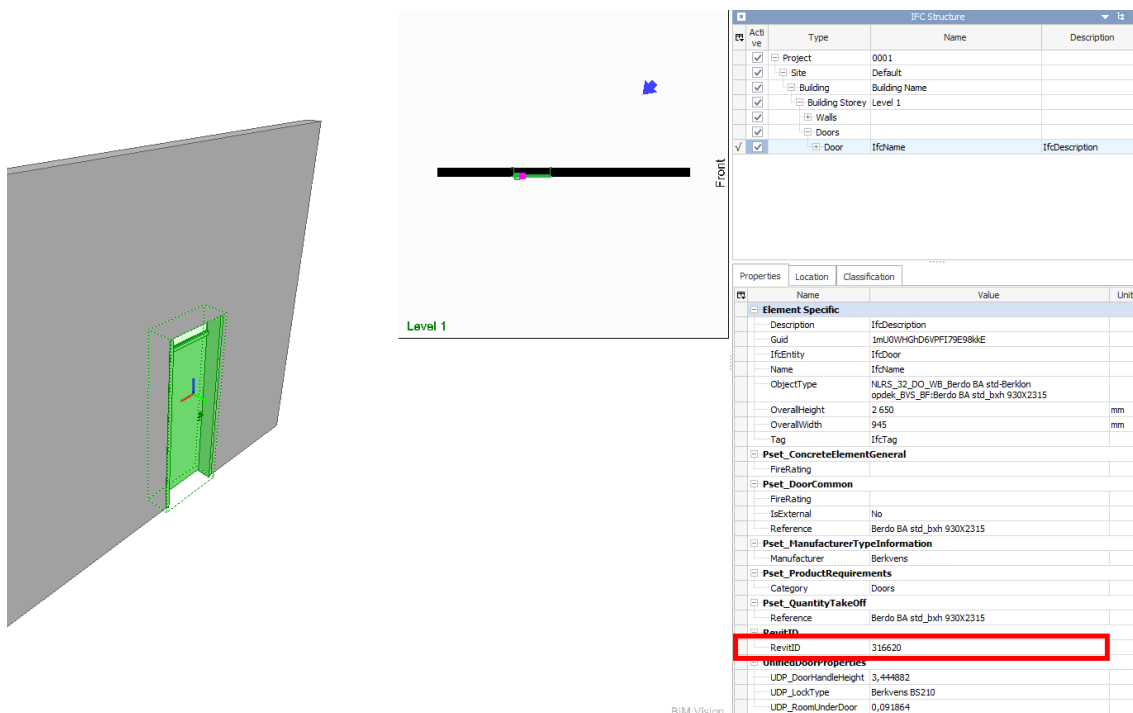
Martijn de Riet, Bimforce Bimforce

```

FixTheDoors_PSet.txt - Kladblok
Bestand  Bewerken  Opmaak  Beeld  Help

#      NominalWidth      Real      COBie.Type.NominalWidth
#      NominalHeight     Real      COBie.Type.NominalHeight
#      Shape              Text       COBie.Type.Shape
#      Size               Text       COBie.Type.Size
#      Color              Text       COBie.Type.Color
#      Finish             Text       COBie.Type.Finish
#      Grade              Text       COBie.Type.Grade
#      Material           Text       COBie.Type.Material
#      Constituents       Text       COBie.Type.Constituents
#      Features           Text       COBie.Type.Features
#      AccessibilityPerformance Text     COBie.Type.AccessibilityPerformance
#      CodePerformance    Text       COBie.Type.CodePerformance
#      SustainabilityPerformance Text     COBie.Type.SustainabilityPerformance
#
PropertySet:  UnifiedDoorProperties  I      IfcDoor
  UDP_RoomUnderDoor      Real      60_66 Ruimte onder de deur
  UDP_RoomUnderDoor      Real      ROD
  UDP_DoorHandleHeight   Real      80_30 Krukgaathoogte
  UDP_DoorHandleHeight   Real      krukgaathoogte
  UDP_LockType           Text       slot
  UDP_LockType           Text       80_40 Merk-type slot

PropertySet:  RevitID I      IfcElement
  RevitID Text      RevitID
  
```



1.5 - More IFC for Dummies

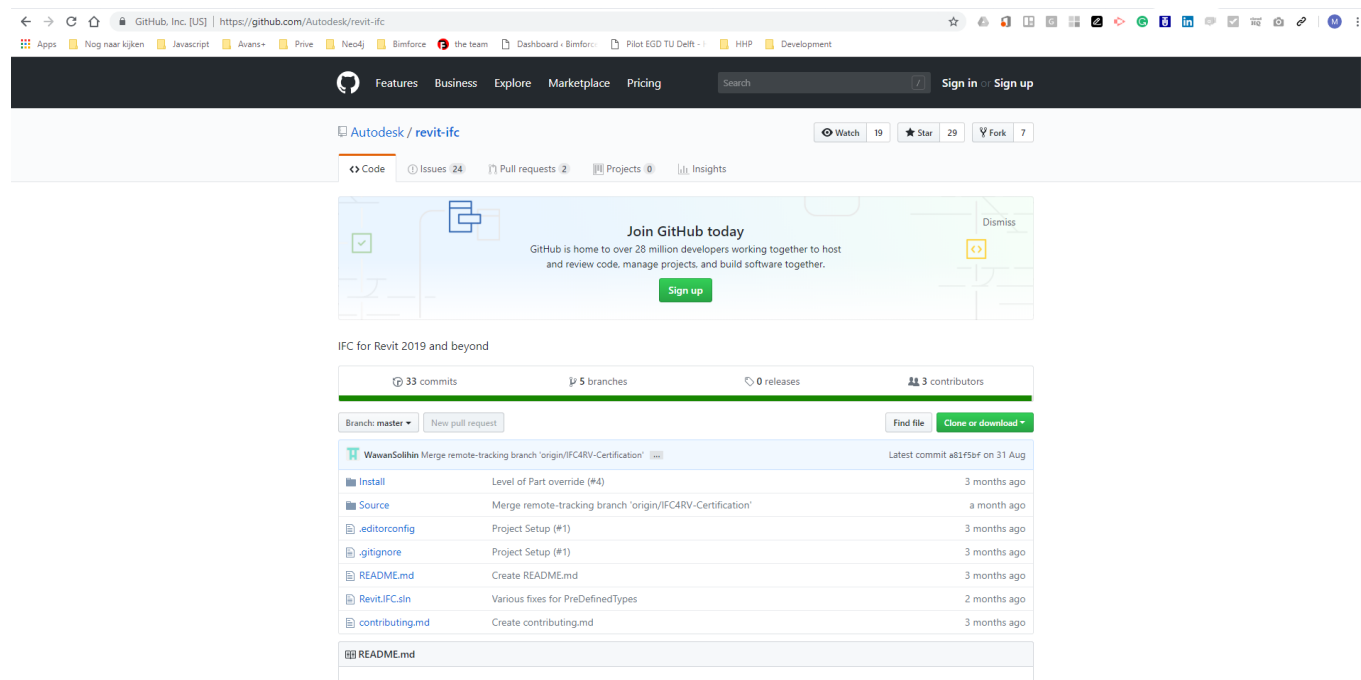
Martijn de Riet, Bimforce Bimforce

Switching to Github

As of recently, the Autodesk Open Source IFC Exporter is no longer developed and maintained on Sourceforge (), but on Github.¹⁰

The reasons for this are mostly based on the desire to increase participation of the community in developing these tools. Github simply provides more extensive possibilities to outside contributors.

The only thing still really missing (and preventing getting of sourceforge completely) is a forum for discussion / interaction with users and developers.



Entity Mapping UI Experiment

In an interesting experiment, Autodesk has been working on an alternative User Interface for IFC mapping in Revit.

This is an effort to improve the hideous workflow with all those pesky textfiles as discussed earlier in this chapter.

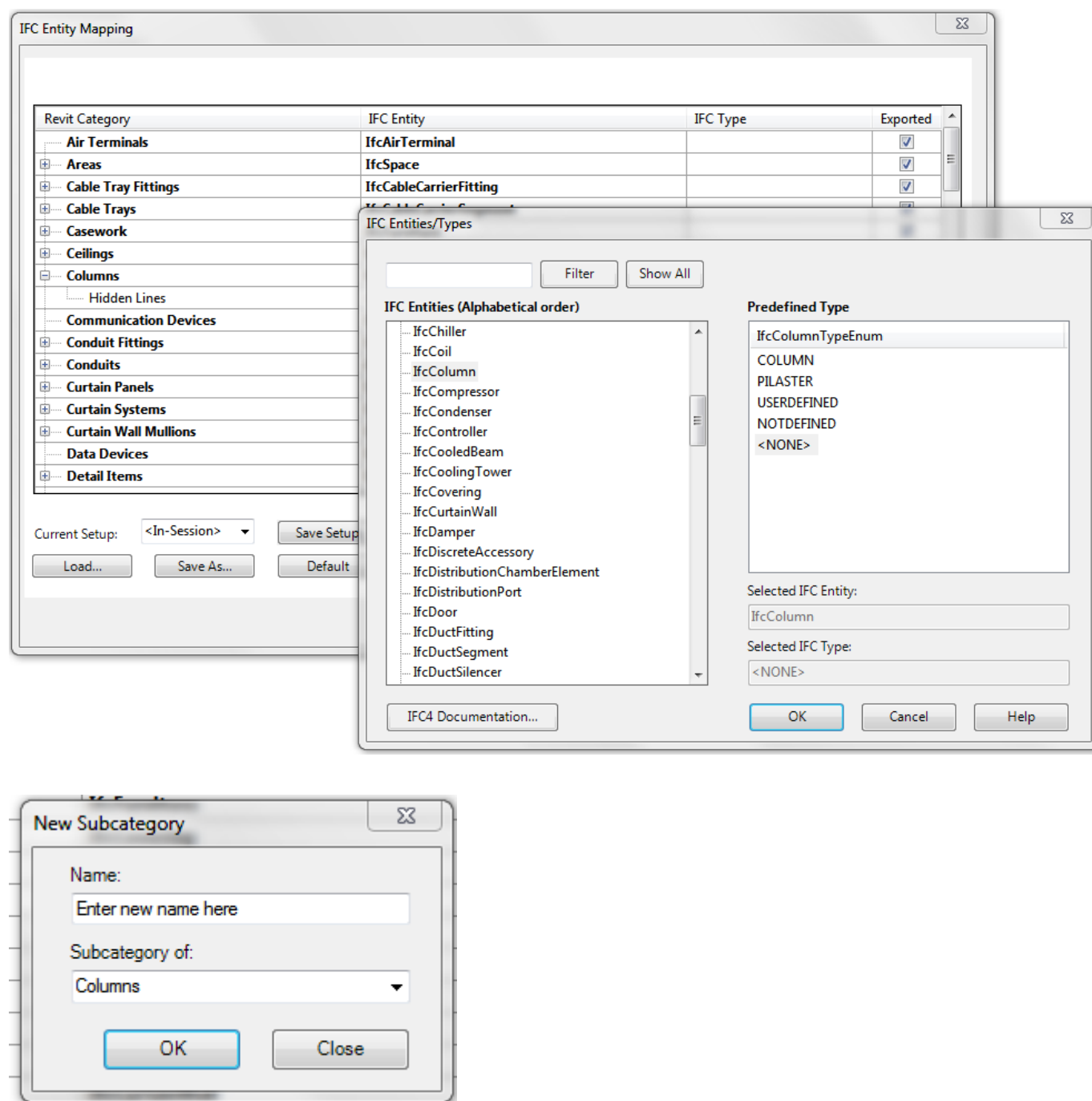
You can download it from the Sourceforge page (at least until they make the transition to Github completely).

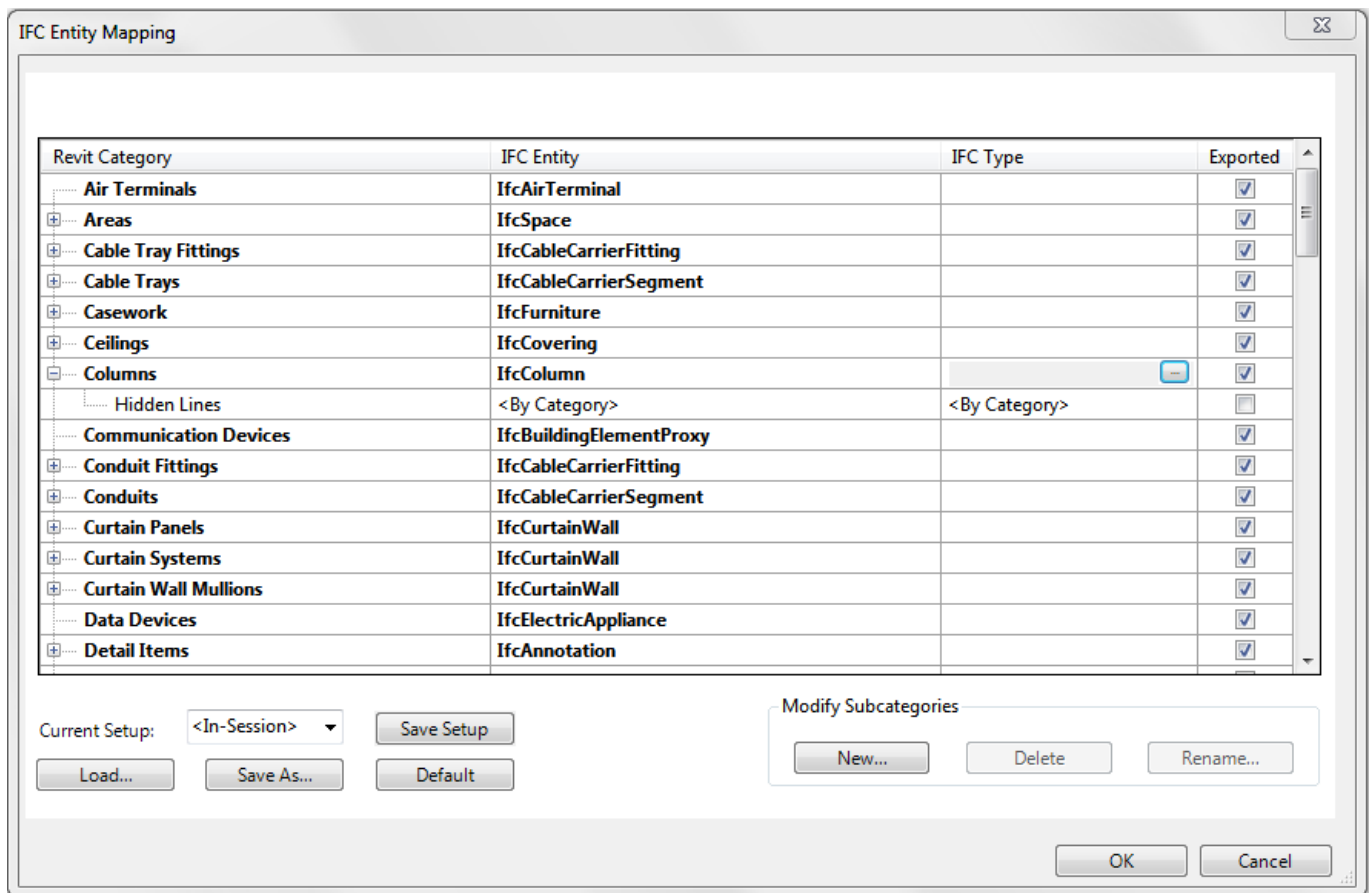
¹⁰ The Github pages for the OS exporter are located here: <https://github.com/Autodesk/revit-ifc>

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

Below you will find some screenshots of the general functionality. However, the general consensus amongst testers is that nobody really got it to function. These screenshots are provided by Autodesk.





It If you would want to see this developed to a larger extent, please contact Autodesk through the Sourceforge or Github pages.¹¹

¹¹ Sourceforge forum for the Open Source Autodesk IFC Exporter:
<https://sourceforge.net/p/ifcexporter/discussion/general/>

An overview of available IFC viewers

Introduction

To begin with, this isn't supposed to be a complete bibliography of all available viewers. So don't get mad if I skip your preferred Viewer. These are just the ones I regularly use and work with. Each has unique trades and I like my viewers specialised.

What they all do have in common is the limited size of the software. I really don't like the 100+mb Viewers floating around (up to a 2.3gb filesize for some). An IFC Viewer should be lightweight and easy to use. As an example: all viewers I've listed in this chapter combined are smaller than the Solibri Model Viewer.

Areddo

Little known viewer from a Dutch company called Arkey Systems. The USP for Areddo is simply that it's a fast, lightweight viewer that you can basically email to your clients to work with. The installer is 5.4mb without having to download additional content. And that is starting to get big, because the first version was under 1mb.

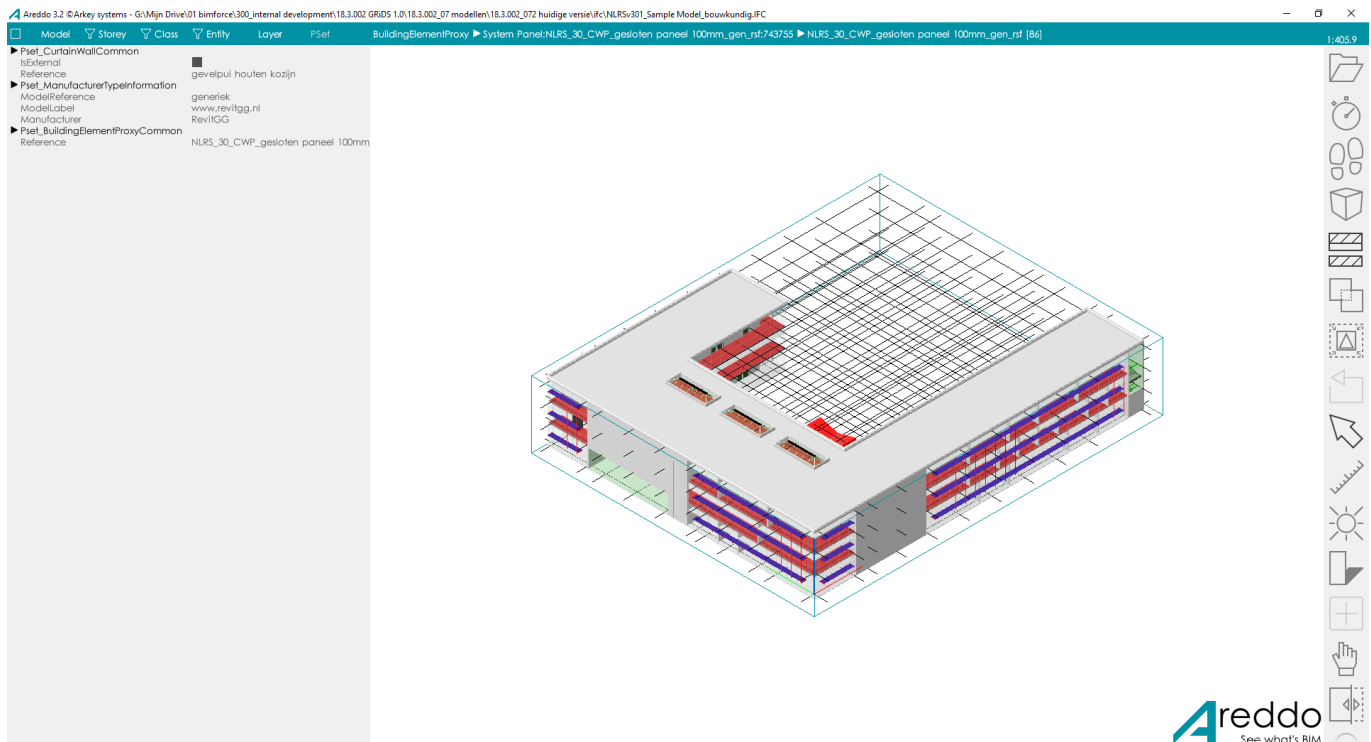
However, given the fact that this extra 4mb provides with support for multiple file formats, basic clash detection and some other cool stuff, it is forgiven.

The one downside for this viewer: it has a completely different kind of navigation. I'm still not convinced I like it. But I love the audacity and attention to detail. This viewer just works.

1.5 - More IFC for Dummies



Martijn de Riet, Bimforce Bimforce



I've added the installer to the class handout. It does require you to go into the Program Files (x86) folder and create your own shortcut. They simply cut out the entire fancy installer and file menu. You double-click the executable, select a file and it loads in the viewer.¹²

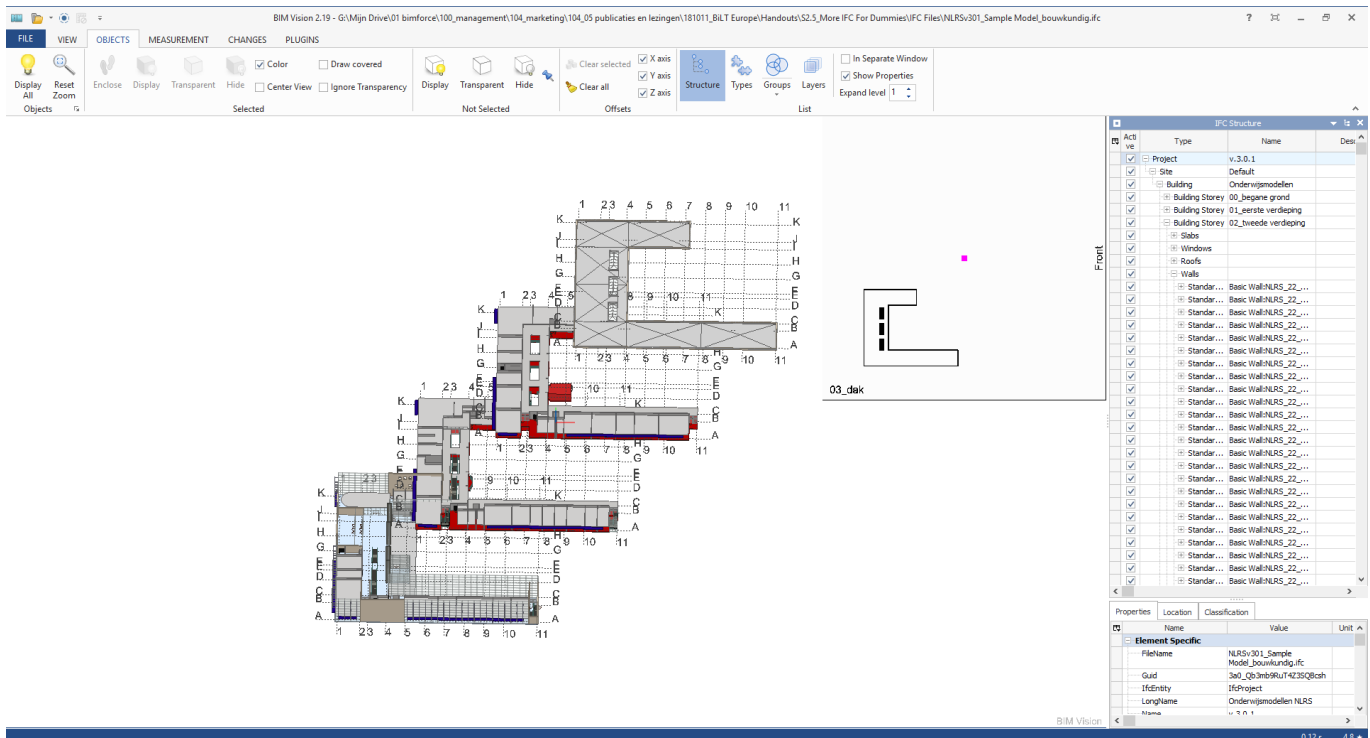
BIM Vision

This one is slightly bigger, but still just 22mb. The extra weight is simply because this software is more developed as a traditional Windows application. But BIM Vision packs one great extra feature: it comes with an API. There are plugins available for all kinds of additional features and workflows. Also worth noting: this is one of 2 viewers I know of that actually supports IFC4 (the other one being the FZK Viewer, which you can download here: <https://www.iai.kit.edu/1302.php>)

¹² You can download the installer here: <http://www.aredo.com/download/> and find tutorials here: <http://www.aredo.com/Help/>

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce



I've added the installer to the class handout.¹³

gglfcTreeViewer

A different way of looking at IFC. Developed by Jon Mirtschin, Geometry Gym, it's probably the most straight-forward tool to examine your IFC files.

I like it, because I know where to look. And love to "wander around" in the IFC. If you need to check if certain items are indeed present in the IFC, this is the tool to use.

I've added the installer to the handout. But I would most definitely urge you to visit Jon's website. He has an impressive amount of tools and plugins readily available to transfer IFC files between numerous applications.¹⁴

¹³ You can find documentation about BIM Vision here: <https://bimvision.eu/en/functionality/>, and tutorial videos here: <https://bimvision.eu/en/video/>

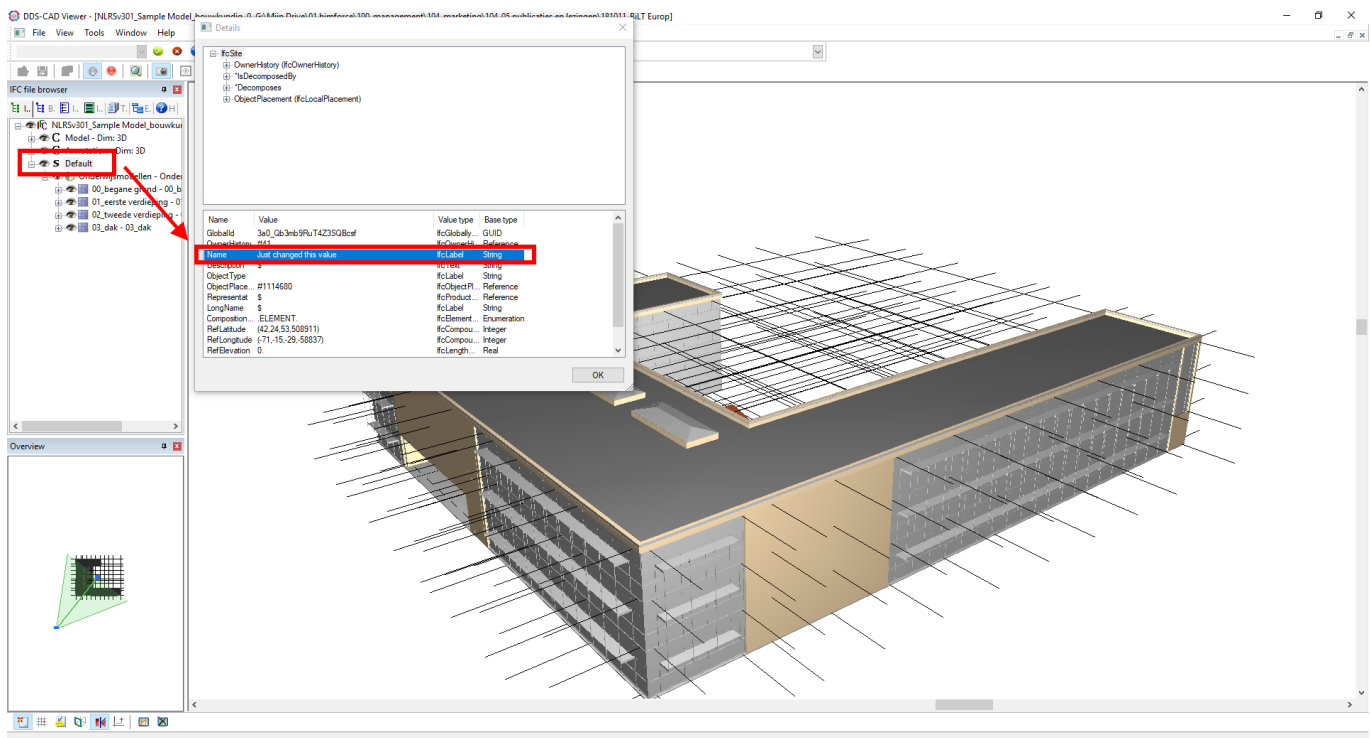
¹⁴ Visit the GeometryGym download section here: <https://geometrygym.wordpress.com/downloads-windows/>

```
GeometryGym IFC Tree Browser http://www.geometrygym.com  
[ Search Id ] [ Include Entity Size ]  
  
IFC  
├─ #109=IFCPROJECT('3a0_Qb3mb9RtZ4Z3SOBcsh'#41;v.3.0.1'$.$,'Onderwijsmodellen NLR$','DEFINITIEF'(#97#105)#92);  
│   ├── IsDecomposedBy: #1114681=IFCSITE('3a0_Qb3mb9RtZ4Z3OBScs'#41;'Default.$'#1114680,$$.ELEMENT.(42,24.53.508911),(71:-15;-29.-58837),0.0,$$);  
│   │   └─ Placement: #1114680=IFCLOCALPLACEMENT('#1114679);  
│   │       └─ RelativePlacement: #1114679=IFCAXIS2PLACEMENT3D(#6,$$$);  
│   ├── IsDecomposedBy: #118=IFCBUILDING('3a0_Qb3mb9RtZ4Z3OBScs'#41;'Onderwijsmodellen'$$.#32,$$.Onderwijsmodellen'.ELEMENT.$$.#114);  
│   │   └─ Placement: #32=IFCLOCALPLACEMENT('#114680,$$);  
│   └─ IsDecomposedBy:  
│       └─ IsDecomposedBy:  
│           └─ #127=IFCBUILDINGSSTOREY('3a0_Qb3mb9RtZ4Z3VbqPS#'41;'00_begane grond'$$.#125,$$.00_begane grond'.ELEMENT.0.0);  
│               └─ ContainsElements(181)  
│                   ├── #7876=IFCSLAB('00JZQhJdVmRCERFFQ4mqp'#41;'Floor.NLRS_23_FL_generiek-100mm_gen_rsf73596'#41;'Floor.NLRS_23_FL_generiek-100mm_gen_rsf#750,#784,'73596#';FLOOR);  
│                   ├── #3763=IFCWALLS_STANDARDCASE('1apTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_20_WA_onterpward-300mm_gen_rsf743194'$$.Basic Wall.NLRS_20_WA_onterpward-300mm_gen_rsf30835'#3714,#3735,'743194');  
│                   ├── #3842=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf743195'$$.Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf543552'#3841,$$.743195);  
│                   ├── #9289=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf743196'$$.Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf543552'#9288,$$.743196);  
│                   ├── #14206=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_20_WA_onterpward-300mm_gen_rsf743197'$$.Basic Wall.NLRS_20_WA_onterpward-300mm_gen_rsf30835'#41184,#14204,'743197');  
│                   ├── #14303=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf743198'$$.Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf543552'#14302,$$.743198);  
│                   ├── #16349=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf743199'$$.Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf543552'#16348,$$.743199);  
│                   ├── #93393=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf743201'$$.Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf543552'#93318,$$.743201);  
│                   ├── #42147=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf743203'$$.Curtain Wall.NLRS_21_CWA_geveluihout-6x7114mm_gen_rsf543552'#42146,$$.743203);  
│                   ├── #52635=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-56x90mm_gen_rsf743205'$$.Curtain Wall.NLRS_21_CWA_geveluihout-56x90mm_gen_rsf543595'#52634,$$.743205);  
│                   ├── #59717=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-56x90mm_gen_rsf743207'$$.Curtain Wall.NLRS_21_CWA_geveluihout-56x90mm_gen_rsf543595'#59716,$$.743207);  
│                   ├── #65796=IFCURTAINWALL('1qpTsUyH9Kgoor78UFupP'#41;'Curtain Wall.NLRS_21_CWA_geveluihout-56x90mm_gen_rsf743209'$$.Curtain Wall.NLRS_21_CWA_geveluihout-56x90mm_gen_rsf543595'#65795,$$.743209);  
│                   ├── #70359=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf743221'$$.Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf2);  
│                   ├── #70469=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf743222'$$.Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf2);  
│                   ├── #70548=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf743223'$$.Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf2);  
│                   ├── #70627=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf743224'$$.Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf2);  
│                   ├── #70702=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf743225'$$.Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf2);  
│                   ├── #70753=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_20_WA_onterpward-200mm_gen_rsf743226'$$.Basic Wall.NLRS_20_WA_onterpward-200mm_gen_rsf30702'#70731,'743226');  
│                   ├── #70830=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_20_WA_onterpward-300mm_gen_rsf743227'$$.Basic Wall.NLRS_20_WA_onterpward-300mm_gen_rsf30835'#70828,'743227');  
│                   ├── #70973=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf743228'$$.Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf2);  
│                   ├── #71032=IFCWALLS_STANDARDCASE('1qpTsUyH9Kgoor78UFupP'#41;'Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf743229'$$.Basic Wall.NLRS_22_WA_ms-wand gips isolate gips-125mm 125-125-125_gen_rsf2);
```

If I have a default viewer, this is it. Simple, robust and not too picky about following all IFC rules. And it provides understandable error messages if it does encounter a problem. Something it holds over any other IFC viewer I know of.

Additionally, DDS Viewer allows you to manually change IFC Properties. Not in any way that would make you want to do it on a regular basis, but still... Pretty sweet! AND, it supports BCF out of the box. Which also makes it very valuable as a stand-alone tool for non-modellers.

¹⁵ You can download the viewer here: <https://www.dds-cad.net/downloads/dds-cad-viewer/>. At the bottom of the page, you will also find a few tutorial videos.



Notepad++

But still, in the end of the day if you need to get your hands dirty, a simple text viewer beats everything. My preferred viewer is Notepad++, simply because (in my experience) it handles large text files well.

So what can you do with something like Notepad++ one might ask? Well, there's a bunch of usecases, but two are of the most common:

1. Check whether an object is exported to the IFC.

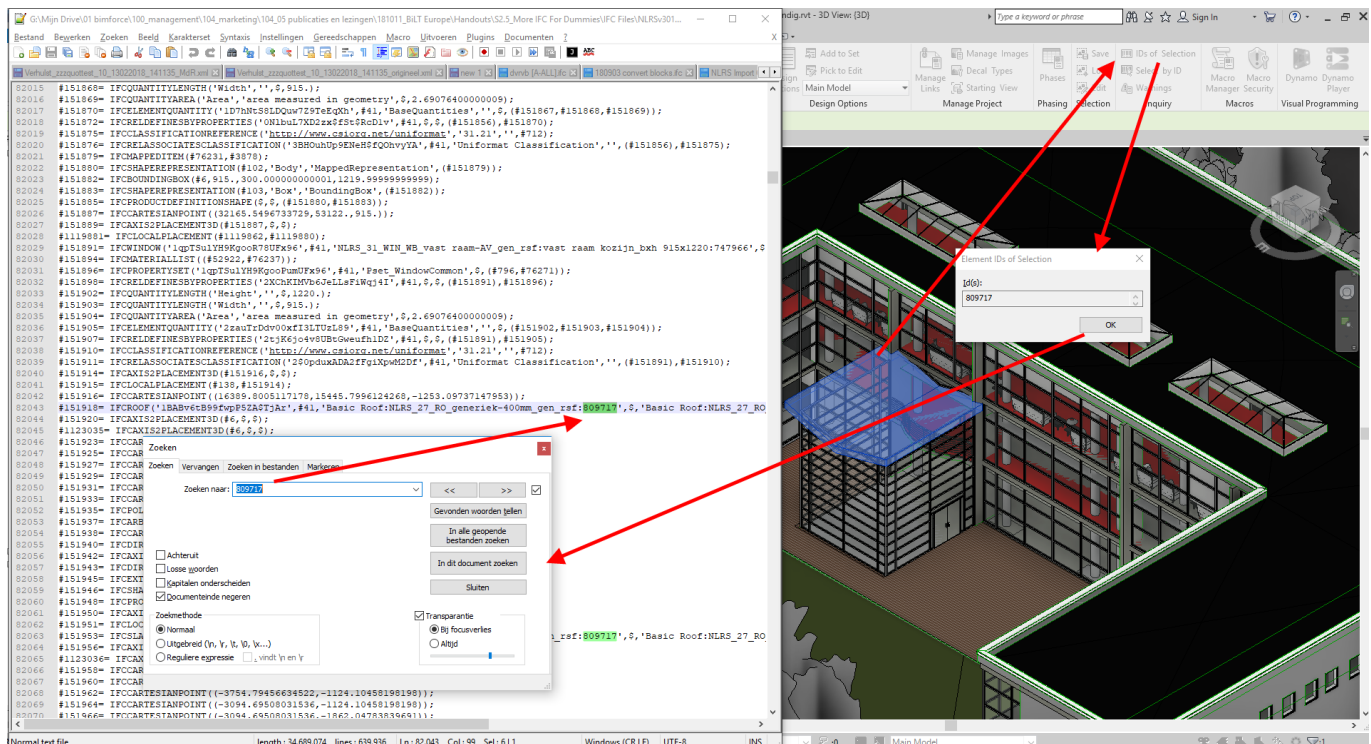
I get into a lot of heated arguments (no, seriously) that basically boil down to a single question: *Is an object exported to IFC or not?*

Usually I will get a call / email from an architectural or engineering firm saying that they are being pummelled by a General Contractor about stuff not showing up in their IFC's. And they don't know how to handle it.

First thing I do then, is request the IFC and Revit file, look up the culprits that aren't showing up in the IFC file, get their Revit ID and open up the IFC in Notepad++. At that point it's a simple Search command (Ctrl+F), paste the Revit ID and see if it pops up. If so, the element is exported and the GC can start looking at their own software to determine why it doesn't show up.

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce



2. Examine extremely large files

Sometimes, things go south. Out of nowhere, you end up with an IFC file which is gigantic in size, even up to 10 times the original file size. The cause of this usually lies within a few Revit Families that the exporter doesn't handle properly. You've seen a few examples of this earlier in this handout.

But if you have just been modelling extensively without keeping a really close look to the stuff that you're dragging into your model, it will get really difficult to examine where things got bloated. At that point you have to start digging into the raw file: which Family is the culprit?

The best way to find out is also the most low-tech solution: open the file in a text editor. When you're there, start looking for a standard series of data mumbo-jumbo:

1. # = IFCCARTESIANPOINT((222.500000000004,199.500000000013,550.));

This part describes a point on a triangle, and always comes in three or more.

2. # = IFCPOLYLOOP((#322,#324,#326,#328));

= IFCFACEBOUND(#333,.T.);

= IFCFACE((#332,#335));

The polyline described here links to 4 cartesian points and draws a line through them.

Then, the Facebound declares the polyline to be the outer perimeter of a face, which is then declared as an actual Face.

3. #1533531= IFCFACETEDBREP(#1533529);

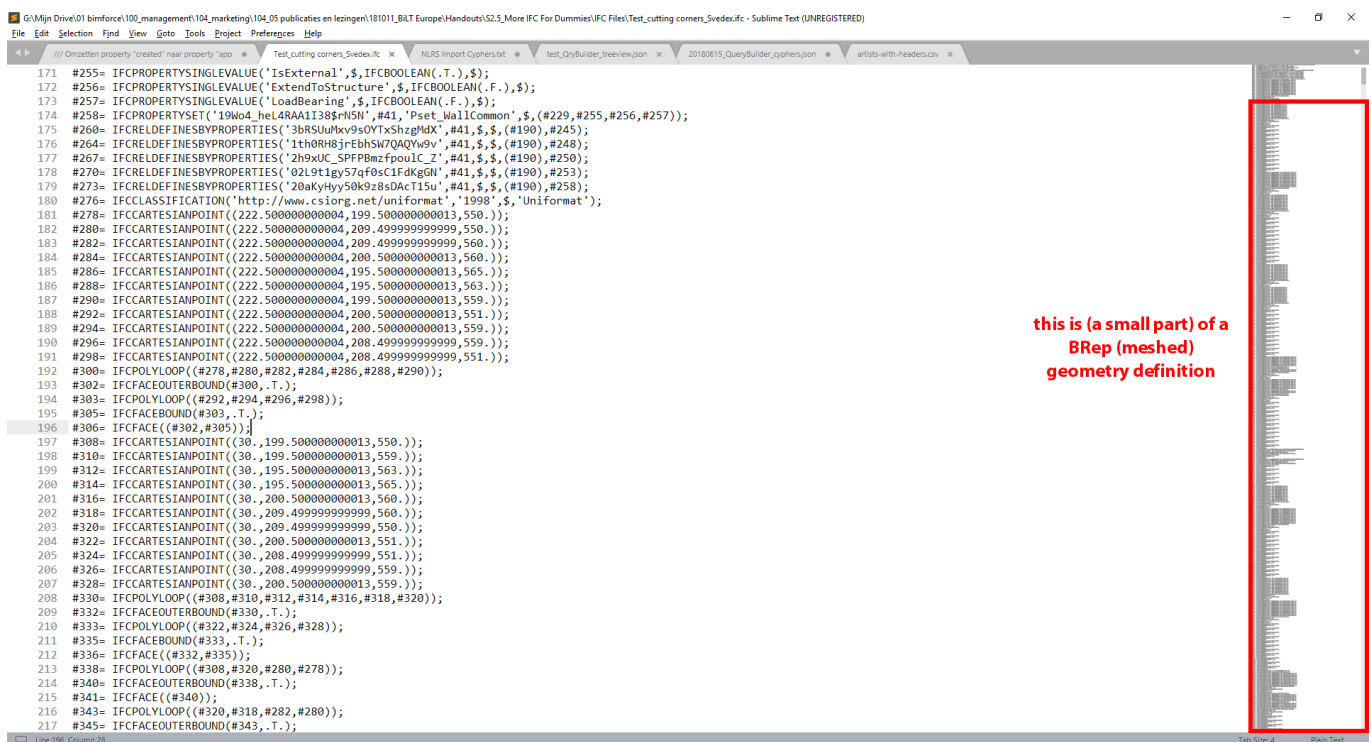
#1533532= IFCSTYLEDITEM(#1533531,(#1533206),\$);

#1533535= IFCCLOSEDSHELL ((#1441137,#1441142,#1441147, and so on

This part stitches all the IfcFace definitions together to make a BRep (or mesh) of the geometry. The last line, IfcClosedShell, can contain thousands of IfcFace mappings and may look something like this:

Now, don't be alarmed when you see some (up to a few hundred at a time) of these lines. We're talking about entire blocks of them. Going up to as much as 100.000 lines of coordinates, polylines and planes defined per instance of an object.

If you look at the example below (taken from the Sublime Text Editor because it shows you this nifty overview), you will see that this example has HUGE chunks of raw geometry data in it.



171 #255= IFCPROPERTYSET('IsExternal', \$, IFCBOOLEAN(T.), \$);
 172 #256= IFCPROPERTYSET('ExtendToStructure', \$, IFCBOOLEAN(F.), \$);
 173 #257= IFCPROPERTYSET('LoadBearing', \$, IFCBOOLEAN(F.), \$);
 174 #258= IFCPROPERTYSET('19Mo4_hel4RAAI385rN5N', #41, 'Pset_WallCommon', \$, (#229, #255, #256, #257));
 175 #260= IFCRELDEFINESBYPROPERTIES('3bRSUuUxv9sOYTxShzgldX', #41, \$, (#190), #245);
 176 #264= IFCRELDEFINESBYPROPERTIES('1th0RH8jrEbh5W7QAQv9v', #41, \$, (#190), #248);
 177 #267= IFCRELDEFINESBYPROPERTIES('2h9uUC_SFPfBmzfpuLC_Z', #41, \$, (#190), #250);
 178 #270= IFCRELDEFINESBYPROPERTIES('02L9t1gV57qf0sC1FdkgGN', #41, \$, (#190), #253);
 179 #273= IFCRELDEFINESBYPROPERTIES('20aKyHy50K9z8sDacT15u', #41, \$, (#190), #258);
 180 #276= IFCCLASSIFICATION('http://www.cs.iong.net/uniformat', '1998', \$, 'Uniformat');
 181 #278= IFCARTESIANPOINT((222.5000000000004, 199.5000000000013, 550.));
 182 #280= IFCARTESIANPOINT((222.5000000000004, 209.4999999999999, 550.));
 183 #282= IFCARTESIANPOINT((222.5000000000004, 209.4999999999999, 560.));
 184 #284= IFCARTESIANPOINT((222.5000000000004, 200.5000000000013, 560.));
 185 #286= IFCARTESIANPOINT((222.5000000000004, 195.5000000000013, 565.));
 186 #288= IFCARTESIANPOINT((222.5000000000004, 195.5000000000013, 563.));
 187 #290= IFCARTESIANPOINT((222.5000000000004, 199.5000000000013, 559.));
 188 #292= IFCARTESIANPOINT((222.5000000000004, 200.5000000000013, 551.));
 189 #294= IFCARTESIANPOINT((222.5000000000004, 200.5000000000013, 559.));
 190 #296= IFCARTESIANPOINT((222.5000000000004, 208.4999999999999, 559.));
 191 #298= IFCARTESIANPOINT((222.5000000000004, 208.4999999999999, 551.));
 192 #300= IFCPOLYLOOP((#278, #280, #282, #284, #286, #288, #290));
 193 #302= IFCFACEOUTERBOUND(#300, .T.);
 194 #303= IFCPOLYLOOP((#292, #294, #296, #298));
 195 #305= IFCFACEBOUND(#303, .T.);
 196 #306= IFCFACE((#302, #305));
 197 #308= IFCARTESIANPOINT((30., 199.5000000000013, 550.));
 198 #310= IFCARTESIANPOINT((30., 199.5000000000013, 559.));
 199 #312= IFCARTESIANPOINT((30., 195.5000000000013, 563.));
 200 #314= IFCARTESIANPOINT((30., 195.5000000000013, 565.));
 201 #316= IFCARTESIANPOINT((30., 200.5000000000013, 560.));
 202 #318= IFCARTESIANPOINT((30., 209.4999999999999, 560.));
 203 #320= IFCARTESIANPOINT((30., 209.4999999999999, 550.));
 204 #322= IFCARTESIANPOINT((30., 200.5000000000013, 551.));
 205 #324= IFCARTESIANPOINT((30., 208.4999999999999, 551.));
 206 #326= IFCARTESIANPOINT((30., 208.4999999999999, 559.));
 207 #328= IFCARTESIANPOINT((30., 200.5000000000013, 559.));
 208 #330= IFCPOLYLOOP((#308, #310, #312, #314, #316, #318, #320));
 209 #332= IFCFACEOUTERBOUND(#330, .T.);
 210 #333= IFCPOLYLOOP((#322, #324, #326, #328));
 211 #335= IFCFACEBOUND(#333, .T.);
 212 #336= IFCFACE((#332, #335));
 213 #338= IFCPOLYLOOP((#308, #320, #280, #278));
 214 #340= IFCFACEOUTERBOUND(#338, .T.);
 215 #341= IFCFACE((#340));
 216 #343= IFCPOLYLOOP((#320, #318, #282, #280));
 217 #345= IFCFACEOUTERBOUND(#343, .T.);

this is (a small part) of a BRep (meshed) geometry definition

[illegible]

This takes you to the first mention of a Property in the IFC file, which always is the definition of parameters belonging to this geometry.

Page 42 of 49

1.5 - More IFC for Dummies



Martijn de Riet, Bimforce Bimforce

Only caveat: this doesn't work when people use IFC Optimizers. These optimizers compress the IFC, but in doing so also mix up the definition. You can no longer trust the properties to come right after the geometry definition, so you would need to start looking at the actual #rule numbers. Which is completely undoable.

Things you didn't know you could do with IFC

Improve your deliverables

I know what you're thinking right now... Did this dude just say "IFC is going to improve your deliverables? What a load of crap!"

And yes, that statement does go into the general perception that IFC gives you, at best and with a LOT of effort, something that is a non-parametric, dumbed-down version of your Revit model. And, to a degree, that's right. There is one big exception though...

Everyone using Revit on projects where a client lays down a hefty Employer Information Requirement (or EIR in short) knows that one of the biggest challenges will be to keep your data structure in place.

So to show you what I'm talking about, let's return to the file Test_cutting_corners.rvt. This file shows perfectly why there's a problem with BIM Content nowadays. Not only do these two Door Families produce a mind-numbing 213 Shared Parameters between them, these are not coordinated in the least.

So how do you get this data nicely formatted? In Revit this would mean opening up one of the Families (or probably both) and swapping out all their parameters with your own Shared Parameters. Which is a drag to do honestly.

Plus, the only thing this accomplishes for you is that you will need to redo this every time you get a new version of the content.

However, when you are required to deliver an IFC, this might just be good for you. Because it's rather simple to clean this mess up. Let's take a few examples:

1. "60-66 Ruimte onder de deur" vs "ROD". Both parameters describe the Room under the Door leaf required for ventilation between rooms.
2. "80-30 Krukgathoogte" vs. "krukgathoogte". Both parameters describe the height for holes in the Door leaf required for the ironmongery.
3. "80-40 Merk-type slot" vs. "NLRs_C_hang en sluitwerk". Both parameters describe the type of ironmongery for the door.

<Door Schedule>					
A	B	C	D	E	F
60_66 Ruimte onder de deur	ROD	80_30 Krukgathoogte	krukgathoogte	80_40 Merk-type slot	slot
	28				SX210_loop
			1050		
28		1050		Merk-type slot : Berkvens BS210	

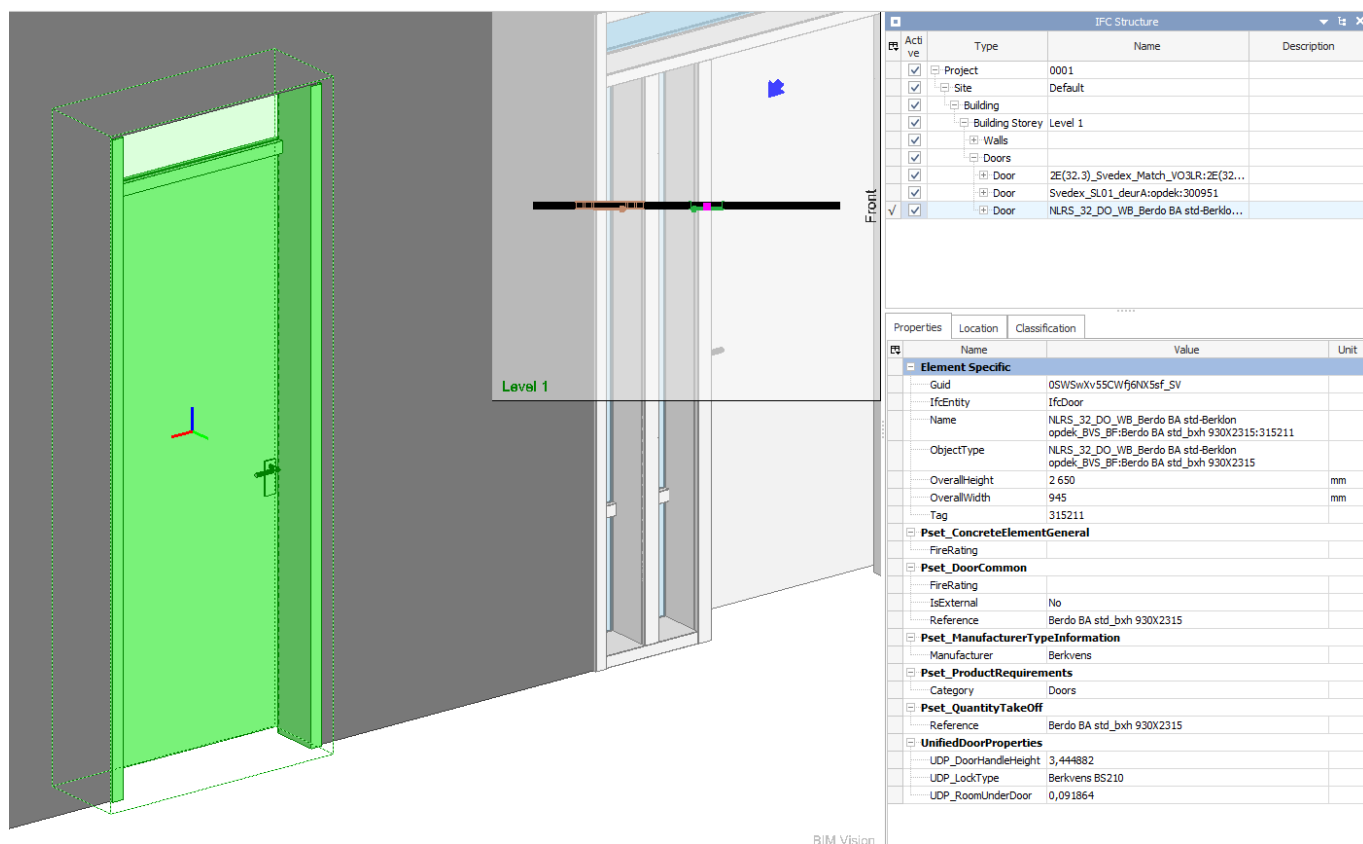
If you want to fix this going forward to IFC you can use the Custom PSet workflow to map these two dataset to a unified IFC Property Set.

```
#
# User Defined PropertySet Definition File
#
# Format:
#   PropertySet: <Pset Name> I[nstance]/T[type] <element list separated by ','>
#   <Property Name 1> <Data type> <[opt] Revit parameter name, if different from IFC>
#   <Property Name 2> <Data type> <[opt] Revit parameter name, if different from IFC>
#   ...
#
# Data types supported: Area, Boolean, ClassificationReference, ColorTemperature, Count, Currency,
#   ElectricalCurrent, ElectricalEfficacy, ElectricalVoltage, Force, Frequency, Identifier,
#   Illuminance, Integer, Label, Length, Logical, LuminousFlux, LuminousIntensity,
#   NormalisedRatio, PlaneAngle, PositiveLength, PositivePlaneAngle, PositiveRatio, Power,
#   Pressure, Ratio, Real, Text, ThermalTransmittance, ThermodynamicTemperature, Volume,
#   VolumetricFlowRate
#
# Example property set definition for COBie:
#
#PropertySet:      COBie_Specification      T      IfcElementType
#   NominalLength      Real      COBie.Type.NominalLength
#   NominalWidth      Real      COBie.Type.NominalWidth
#   NominalHeight      Real      COBie.Type.NominalHeight
#   Shape      Text      COBie.Type.Shape
#   Size      Text      COBie.Type.Size
#   Color      Text      COBie.Type.Color
#   Finish      Text      COBie.Type.Finish
#   Grade      Text      COBie.Type.Grade
#   Material      Text      COBie.Type.Material
#   Constituents      Text      COBie.Type.Constituents
#   Features      Text      COBie.Type.Features
#   AccessibilityPerformance      Text      COBie.Type.AccessibilityPerformance
#   CodePerformance      Text      COBie.Type.CodePerformance
#   SustainabilityPerformance      Text      COBie.Type.SustainabilityPerformance
#
#
#PropertySet:      UnifiedDoorProperties      I      IfcDoor
#   UDP_RoomUnderDoor      Real      60_66 Ruimte onder de deur
#   UDP_RoomUnderDoor      Real      ROD
#   UDP_DoorHandleHeight      Real      80_30 krukgethoogte
#   UDP_DoorHandleHeight      Real      krukgethoogte
#   UDP_LockType      Text      slot
#   UDP_LockType      Text      80_40 Merk-type slot
```

Btw, a pro-tip:

The hashtags (#) mean the line is commented and therefor ignored by Revit. This also applies to other text files, including the Shared Parameter File.

When all this goes according to plan, you should see something like this:



And here you are, a cleaned-up, improved version of your Revit model.

Parametric, cross-platform BIM Content

One of the greatest myths about IFC is that it is not capable of handling parametrics. In other words: IFC is always a static, dumb blob of data, with no real intelligence to it.

That is flat-out wrong.

There's no other way to put it. You **can** use IFC to define parametric objects. That would work exactly as you expect them to in Revit. With parametric constraints, connectors, all types of data and everything.

The upside is: these IFC definitions don't just work in Revit, they work in other software too.

During the presentation I will be showcasing a few examples of how this should work. Describing the fundamentals behind it, or the technical implementation would be a complete class on its own. However, if you've seen the presentation (or this handout)

1.5 - More IFC for Dummies

Martijn de Riet, Bimforce Bimforce

and want to know more: reach out on Twitter/Linkedin, drop me an email or give me a call and I'll be happy to discuss it further.

Revit – Excel – IFC workflows

Let's start with a HUGE shoutout to the IFC Wizard himself, Jon Mirtschin. Jon created this workflow as part of a client project. We at Bimforce just picked it up to enhance our own IFC-based solutions and workflows.

Why? You mean besides the fact that this is an utterly awesome development? Well, simply because this perfectly matches our philosophy at Bimforce: utilising IFC to improve AECOM processes without the tedious, migraine-inducing, academic-level highly technical non-user-friendly workflows that we usually associate with working with IFC...

I will be dedicating a large part of the class on this subject, so I'm not going to spoil it with writing down the specifics in this handout. If you're interested in how you would be able to work with Excel and IFC generators to dynamically generate your building (content) in Revit you have no choice but to come to Session 1.5

Resources

Throughout this handout I have referenced several other publications. The list below provides a summary of all these documents. Naturally I would like to insist that you give proper credit when using these, and respect any and all IP-restrictions that might come with these resources.

1. Shoutout to this awesome website, for anyone interested in the Revit API:
<http://www.revitapidocs.com/2018/>
2. Oh dreaded IFC2x3 documentation:
<http://www.buildingsmart-tech.org/ifc/IFC2x3/TC1/html/>
3. The slightly less dreaded IFC4 documentation:
<http://www.buildingsmart-tech.org/ifc/IFC4/Add2/html/>
4. More on the history of BuildingSMART here:
<https://www.buildingsmart.org/about/about-buildingsmart/history/>
Videos "Tower of Babel", pt1 and 2 explaining the idea behind IFC:
https://www.youtube.com/watch?v=g_jmGQvr6dQ and
<https://www.youtube.com/watch?v=vTyB96O7Xeg>
5. Besides the resources from our website, you might want to check this documentation out: <http://www.buildingsmart-tech.org/ifc/>
6. Find the IFC CV 2.0 wallpaper through the following link:
http://www.buildingsmart-tech.org/downloads/view-definitions/coordination-view/sub-schema/CoordinationView_V2-0_WallPaper_IFC2X3_Version-1.pdf/view
7. The Autodesk Revit IFC Manual can also be found here:
http://abcdblog.typepad.com/abcd/2018/Success_Stories/IFC-Manual-2018-ENU.PDF
8. Sourceforge post on how to override Project, Building and Site Guides:
<https://sourceforge.net/p/ifcexporter/discussion/general/thread/634a2a16/>
9. Sourceforge post on how to use expressions:
<https://sourceforge.net/p/ifcexporter/wiki/Notes%20on%20parameter%20expression/>
10. The Github pages for the OS exporter are located here:
<https://github.com/Autodesk/revit-ifc>
11. Sourceforge forum for the Open Source Autodesk IFC Exporter:
<https://sourceforge.net/p/ifcexporter/discussion/general/>
12. Areddo IFC viewer download en help files:
<http://www.aredo.com/download/>
<http://www.aredo.com/Help/>
13. Bimvision documentation:
<https://bimvision.eu/en/functionality/> and <https://bimvision.eu/en/video/>

14. Visit the GeometryGym download section here for all his tools (including the Treeview):
<https://geometrygym.wordpress.com/downloads-windows/>
15. Download the DDSCAD viewer and tutorials here:
<https://www.dds-cad.net/downloads/dds-cad-viewer/>.